School of Technology, Design & Computer Application
College of Technology
B. TECH (CE/CSE)
Full Stack Web Development

Semester:	6th	**Academic Year:**	2025 - 26
Course Name:	FSWD	**Course Code:**	1010043350

Question bank

Sr. No.	**Question Text**
Unit No 1: Introduction to HTML5, Tailwind CSS	
1	Explain the key features of HTML 5.
2	What are the new semantic elements introduced in HTML5?
3	Explain the use of multimedia elements in HTML5 with proper example.
4	Identify new attributes that enhanced HTML5 forms.
5	Demonstrate the use of <canvas> and <svg> tag in HTML 5
6	What are the deprecated features in HTML5, and why were they removed?
7	Explain the role of Graphics and Media APIs in HTML5 with proper examples.
8	What is Tailwind CSS? how is it different from traditional CSS? Discuss the benefits of it.
9	What are the methods to install Tailwind CSS in a project? Explain it.
10	Discuss the significance of layout and background utilities in Tailwind CSS with examples.
11	Describe the use of Flexbox in Tailwind CSS.
12	Demonstrate the use of spacing, sizing, and alignment utilities in Tailwind CSS with use cases.
13	Explain how to create animations using Tailwind CSS.
14	Explain the role of transformations and SVG utilities in creating interactive UI designs.
15	How can you manage spacing and alignment using Tailwind CSS?
16	Write SVG tag to draw a rectangle, triangle and circle filled with blue color and a red border.
17	Discuss the new enhancements in HTML5 forms.
18	How can you create a responsive grid layout using Tailwind CSS?
Unit No 2 : Introduction to React JS	
19	What are the key features of React JS?
20	Describe the process of setting up a React environment.

21	What is component in React JS? Explain different types of components with example.
22	Explain functional components in React JS with proper example.
23	What is the difference between React Props and States?
24	How can you pass data from a parent component to a child component using props?
25	What are React Hooks? Explain commonly used React Hooks with proper example.
26	Explain the useState Hook with an example.
27	How does React Router help in building single-page applications?
28	Explain concept of Routing in React JS with proper example
29	How is form validation handled in React JS? Explain with proper example.
30	Describe the role of the<BrowserRouter> , <Routes> and <Route> components in React Router.
31	How does React Router manage navigation between different pages in a single-page application (SPA)?
32	Write a React code that connect with PHP and retrieve data using useEffect.
33	How React.js and PHP Work Together? Explain with proper example
Unit No 3: Introduction to PHP 8	
34	Explain the new features introduced in PHP 8
35	What is Just-In-Time (JIT) compilation in PHP 8?
36	Explain the purpose of the Nullsafe operator in PHP 8.
37	What are Union Types in PHP 8, and how are they used?
38	What are Named Arguments? Explain with proper example.
39	Explain Constructor Property Promotion in PHP 8.
40	What is the Match expression in PHP 8, and how does it differ from a switch statement?
41	Describe how to build RESTful APIs using PHP 8.
42	What are the benefits of using GraphQL with PHP 8?
43	How is JWT authentication implemented in PHP 8?
44	Explain the concept of attributes (annotations) in PHP 8.
45	Explain the core principles of Object-Oriented Programming in PHP with examples.
46	How do PHP classes and objects work? Illustrate with an example
47	How can the Stringable interface be used in PHP 8?
Unit 4 : Database Connectivity Using MongoDB	
48	What are the key features of MongoDB?
49	When to use MongoDB ?
50	How do you create a database and collection in MongoDB?
51	What are the various methods available in MongoDB for performing CRUD operations? Explain with proper example.
52	What are query operators in MongoDB? Explain with proper example.
53	Explain MongoDB comparison and logical operators with proper example.
54	What is Aggregation? Explain Aggregation in MongoDB with proper example.

55	How can we connect PHP with MongoDB? Explain in details.
56	Write a PHP code to perform CRUD Operations (insert, update, delete, read) with PHP & MongoDB
57	What is indexing? How is indexing implemented in MongoDB?
58	How can data validation be performed in MongoDB?
59	Explain how to use the MongoDB Query API.
60	What is the purpose of MongoDB Charts?
Unit 5 : Introduction About Laravel	
61	What is Laravel? Explain its key features and benefits.
62	Describe the Model-View-Controller (MVC) architecture. How does Laravel implement MVC?
63	How do you install Laravel on a local machine? Explain the steps.
64	What is Composer, and why is it required for Laravel installation?
65	What are Laravel Artisan commands? Provide examples of commonly used commands.
66	Explain the directory structure of a Laravel project in details.
67	Explain the concept of routing in Laravel.
68	What are Controllers in Laravel, and how do they work?
69	How do you pass data from a Controller to a View? Provide an example.
70	Explain the concept of Blade Templating in Laravel. What are its benefits?
71	How do you retrieve input data from a form submission using Laravel Request?
72	How do you create and handle Forms in Laravel?
73	How do you create and manipulate Laravel Collections? Provide examples.
74	What is Eloquent ORM in Laravel? How does it simplify database interactions? Discuss the benefits of it.
75	What are Packages in Laravel? How to install it? Discuss Popular Laravel Packages.

Course Coordinator **Head of Department**

Silver Oak University

Aditya Silver Oak Institute of Technology

School of Technology, Design & Computer Application

Full Stack Web Development

Question Bank — Complete Solutions

All 5 Units | 75 Questions | Full Coverage

Field	Details
Course Name	Full Stack Web Development
Course Code	1010043350
Semester	6th Semester
Branch	B.Tech CSE / CE
Academic Year	2025 – 26
Institute	Aditya Silver Oak Institute of Technology
University	Silver Oak University, Ahmedabad
Enrollment No.	2302031800037

Unit	Topic	Questions
Unit 1	Introduction to HTML5 & Tailwind CSS	Q1 – Q18
Unit 2	Introduction to React JS	Q19 – Q33
Unit 3	Introduction to PHP 8	Q34 – Q47
Unit 4	Database Connectivity Using MongoDB	Q48 – Q60
Unit 5	Introduction to Laravel	Q61 – Q75

UNIT 1: Introduction to HTML5 & Tailwind CSS

Q1. Explain the key features of HTML5.

HTML5 is the fifth and latest major version of the HyperText Markup Language, officially standardized by the W3C. It introduced a wide range of new features that made the web richer, more interactive, and more accessible — without relying on third-party plugins like Flash. HTML5 is backward compatible, meaning old HTML4 code still works within HTML5 browsers.

Key Features of HTML5:

- Semantic Elements: New tags like <header>, <footer>, <article>, <section>, <nav>, and <aside> that give meaningful structure to web pages. These help search engines and screen readers understand the content better.
- Multimedia Support: Native <audio> and <video> tags allow embedding media without Flash or third-party plugins. Supports MP4, WebM, Ogg formats.
- Canvas API: The <canvas> element allows dynamic 2D drawing using JavaScript — useful for games, charts, and animations.
- SVG Support: Inline Scalable Vector Graphics allow resolution-independent graphics directly in HTML.
- Geolocation API: Allows websites to request the user's location (with permission) using navigator.geolocation.
- Web Storage: localStorage and sessionStorage provide client-side storage without cookies, offering better performance and capacity (up to 10 MB).
- Web Workers: Enable background JavaScript execution in separate threads without blocking the main UI thread.
- WebSockets: Provides full-duplex communication channels for real-time applications like chats and live dashboards.
- Drag and Drop API: Native drag-and-drop support for interactive UI without JavaScript frameworks.
- Improved Forms: New input types (email, date, range, color, search), attributes (placeholder, required, autofocus), and built-in validation.
- Offline Support: The Application Cache (now replaced by Service Workers) allowed pages to work offline.

Summary Table:

Feature	Description
Semantic Tags	<header>, <footer>, <section>, <article>, <nav>
Multimedia	<audio>, <video> without Flash
Canvas	2D graphics via JavaScript
Web Storage	localStorage / sessionStorage
Geolocation	Access user location via API
Web Workers	Background JavaScript threads
Forms	New input types and validation attributes

Q2. What are the new semantic elements introduced in HTML5?

Semantic elements are HTML tags that clearly describe their meaning and purpose to both the browser and the developer. Before HTML5, developers used generic and tags with class names like class='header' or class='nav', which carried no structural meaning. HTML5 introduced semantic tags that explicitly define the role of the content inside them.

New Semantic Elements in HTML5:

Element	Purpose	Example Use
<header>	Represents the top section of a page or section	Site logo, navigation
<footer>	Bottom section with copyright, links	Footer links, credits
<nav>	Navigation links block	Menu bar, sidebar links
<main>	Main content of the page (unique per page)	Primary article content
<section>	Thematic grouping of content	A chapter or topic group
<article>	Self-contained piece of content	Blog post, news article
<aside>	Side content related to main content	Sidebar, ads, tips
<figure>	Self-contained illustration or media	Image with caption
<figcaption>	Caption for a <figure>	Image description text
<mark>	Highlighted text	Search result highlighting

<time>	Machine-readable date/time	Event dates
<details>	Disclosure widget (show/hide)	FAQ accordions
<summary>	Summary label for <details>	FAQ question label

Example usage:

```
<!DOCTYPE html>
<html>
  <header>
    <h1>My Website</h1>
    <nav>
      <a href='/home'>Home</a>
      <a href='/about'>About</a>
    </nav>
  </header>
  <main>
    <article>
      <h2>Blog Post Title</h2>
      <p>Content of the blog post...</p>
    </article>
    <aside>
      <p>Related links and sidebar content</p>
    </aside>
  </main>
  <footer>
    <p>&copy; 2025 My Website</p>
  </footer>
</html>
```

Advantages of Semantic Elements: Better SEO (search engines understand page structure), better accessibility (screen readers navigate properly), cleaner code that is easier to maintain and read.

Q3. Explain the use of multimedia elements in HTML5 with proper example.

HTML5 introduced native support for multimedia — audio and video — directly in the browser without requiring Flash Player or any third-party plugin. The two primary multimedia elements are <audio> and <video>. Both elements support multiple source formats to ensure cross-browser compatibility.

1. The <video> Element:

Used to embed video content in a webpage. Supports MP4, WebM, and Ogg formats.

```
<video width="640" height="360" controls autoplay muted loop>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

Key attributes of <video>:

Attribute	Description
controls	Shows play, pause, volume controls
autoplay	Starts playing automatically on load
muted	Mutes audio (required for autoplay in Chrome)
loop	Replays video when it ends
poster	Image displayed before video plays
width / height	Dimension of the video player

2. The <audio> Element:

Used to embed audio content. Supports MP3, WAV, and Ogg formats.

```
<audio controls>
  <source src="music.mp3" type="audio/mpeg">
  <source src="music.ogg" type="audio/ogg">
  Your browser does not support the audio tag.
</audio>
```

3. Complete Multimedia Webpage Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>HTML5 Multimedia Demo</title>
</head>
<body>
  <h2>Watch the Tutorial</h2>
  <video width="600" controls poster="thumbnail.jpg">
    <source src="tutorial.mp4" type="video/mp4">
    Your browser does not support video.
  </video>
  <h2>Listen to the Podcast</h2>
  <audio controls>
    <source src="podcast.mp3" type="audio/mpeg">
    Your browser does not support audio.
  </audio>
</body>
</html>
```

These elements work across all modern browsers — Chrome, Firefox, Safari, and Edge — without any plugin installation, making multimedia integration simple, secure, and responsive.

Q4. Identify new attributes that enhanced HTML5 forms.

HTML5 significantly improved forms by introducing new input types, new attributes, and built-in client-side validation. These enhancements reduce the need for JavaScript-based validation and make forms more user-friendly and accessible.

New Input Types in HTML5:

Input Type	Description	Example
email	Validates email format automatically	<input type="email">
url	Validates URL format	<input type="url">
number	Numeric input with min/max/step	<input type="number" min="1" max="100">
range	Slider control	<input type="range" min="0" max="10">
date	Date picker	<input type="date">
time	Time picker	<input type="time">
color	Color picker palette	<input type="color">
search	Search box with clear button	<input type="search">
tel	Telephone number input	<input type="tel">
week	Week/year picker	<input type="week">

New Form Attributes:

Attribute	Description
placeholder	Shows hint text inside empty input field
required	Field must be filled before submission
autofocus	Auto focuses the field when page loads
autocomplete	Enables/disables browser autocomplete
pattern	Regex validation pattern for the input
min / max	Minimum and maximum values for number/date
step	Step interval for numeric inputs
multiple	Allows multiple values (email, file inputs)
novalidate	Disables form validation on submit
form	Associates input with a form by ID

Complete HTML5 Form Example:

```
<!DOCTYPE html>
<html><body>
<form action="/submit" method="POST">
  <label>Name: <input type="text" placeholder="Enter name" required
autofocus></label><br>
  <label>Email: <input type="email" placeholder="you@example.com"
required></label><br>
  <label>Age: <input type="number" min="18" max="60" step="1"
required></label><br>
  <label>Date of Birth: <input type="date"></label><br>
  <label>Phone: <input type="tel" pattern="[0-9]{10}"></label><br>
  <label>Website: <input type="url" placeholder="https://"></label><br>
  <label>Color: <input type="color"></label><br>
  <button type="submit">Submit</button>
</form>
</html></body>
```

Q5. Demonstrate the use of <canvas> and <svg> tag in HTML5.

HTML5 provides two powerful ways to draw graphics in the browser: the <canvas> element for pixel-based (raster) drawing via JavaScript, and SVG (Scalable Vector Graphics) for resolution-independent vector graphics defined using XML markup.

1. The <canvas> Tag:

The <canvas> element is a drawing surface. It is blank by default — you use JavaScript with the Canvas 2D API to draw on it. It is raster-based, meaning the image is made of pixels and may lose quality when scaled.

```
<!DOCTYPE html>
<html><body>
<canvas id="myCanvas" width="400" height="200" style="border:1px solid
black;"></canvas>
<script>
  // Get the canvas element and its 2D drawing context
  const canvas = document.getElementById("myCanvas");
  const ctx = canvas.getContext("2d");

  // Draw a filled blue rectangle
  ctx.fillStyle = "blue";
  ctx.fillRect(20, 20, 150, 80); // x, y, width, height

  // Draw a red circle
  ctx.beginPath();
  ctx.arc(300, 100, 50, 0, 2 * Math.PI); // x, y, radius
  ctx.fillStyle = "red";
```

```
  ctx.fill();

  // Draw text
  ctx.fillStyle = "white";
  ctx.font = "20px Arial";
  ctx.fillText("Canvas", 50, 70);
</script>
</body></html>
```

2. The <svg> Tag:

SVG stands for Scalable Vector Graphics. It uses XML to define shapes, lines, and text. SVG graphics are resolution-independent — they look sharp at any size.

```
<!DOCTYPE html>
<html><body>
<svg width="400" height="200" xmlns="http://www.w3.org/2000/svg">
  <!-- Rectangle -->
  <rect x="10" y="10" width="120" height="80" fill="blue" stroke="red" stroke-
width="3"/>
  <!-- Circle -->
  <circle cx="250" cy="100" r="60" fill="green" stroke="black" stroke-
width="2"/>
  <!-- Line -->
  <line x1="10" y1="150" x2="390" y2="150" stroke="black" stroke-width="2"/>
  <!-- Text -->
  <text x="50" y="180" font-size="18" fill="black">Hello SVG!</text>
</svg>
</body></html>
```

Canvas vs SVG Comparison:

Feature	Canvas	SVG
Type	Raster (pixel-based)	Vector (XML-based)
Scalability	Loses quality on zoom	Remains sharp at any size
Rendering	JavaScript required	Declarative XML markup
Performance	Better for complex animations/games	Better for static diagrams
DOM	Not part of DOM	Each element is a DOM node
Use Case	Games, real-time charts	Icons, logos, diagrams

Q6. What are the deprecated features in HTML5, and why were they removed?

HTML5 removed or deprecated several older HTML elements and attributes that were considered presentational (styling-related), redundant, rarely used, or harmful to accessibility. The W3C encourages use of CSS for all styling purposes, and HTML should only define structure and semantics. Deprecated elements still exist in old browsers but should no longer be used in modern development.

Deprecated Elements and Reasons:

Deprecated Element	Purpose (Old)	Reason for Removal	Modern Alternative
<font>	Set font face/size/color	Style belongs in CSS	CSS font-family, font-size, color
	Center-align content	Presentational, use CSS	CSS text-align: center
<big>	Bigger text	Presentational	CSS font-size
<small>	Smaller text (now redefined)	Overused for styling	CSS / semantic use
<strike>	Strikethrough text	Use <del> or <s>	<del>, <s> or CSS
<u>	Underline text (reused)	Caused confusion with links	CSS text-decoration or <ins>
<frame> / <frameset>	Split page into frames	Accessibility nightmare, SEO issues	CSS layout, iframes
<noframes>	Fallback for frames	Frames deprecated entirely	Not applicable
<acronym>	Abbreviations	Redundant — use <abbr>	<abbr title='...'>
<applet>	Embed Java applets	Security risk, plugin-dependent	<object> or web APIs
bgcolor attribute	Background color on elements	Style belongs in CSS	CSS background-color
align attribute	Alignment on elements	Presentational	CSS text-align / flexbox

Why Were They Removed?

- Separation of Concerns: HTML should define structure; CSS should handle presentation. Mixing both makes maintenance difficult.
- Accessibility: Elements like <frame> broke keyboard navigation and screen readers.
- Security: <applet> allowed execution of potentially malicious Java code.
- Redundancy: Multiple elements did the same job (e.g., <acronym> vs <abbr>).
- Performance: Framesets created multiple HTTP requests and complex navigation.

Q7. Explain the role of Graphics and Media APIs in HTML5 with proper examples.

HTML5 introduced several Graphics and Media APIs that allow developers to create rich interactive experiences directly in the browser — without Flash or other plugins. These APIs are accessed using JavaScript and work with HTML5 elements like <canvas>, <video>, and <audio>.

1. Canvas 2D API:

Used for drawing 2D shapes, text, images, and animations. Accessed via the getContext('2d') method on a <canvas> element.

```
<canvas id="c" width="300" height="150"></canvas>
<script>
  const ctx = document.getElementById("c").getContext("2d");
  ctx.fillStyle = "orange";
  ctx.fillRect(10, 10, 200, 100); // Draws filled rectangle
  ctx.strokeStyle = 'black';
  ctx.lineWidth = 3;
  ctx.strokeRect(10, 10, 200, 100); // Draws border around rectangle
</script>
```

2. WebGL API:

Provides hardware-accelerated 3D graphics rendering. Accessed via getContext('webgl') on canvas. Used in 3D games and visualization tools.

```
<canvas id="gl" width="400" height="300"></canvas>
<script>
  const gl = document.getElementById("gl").getContext("webgl");
  if (gl) {
    gl.clearColor(0.1, 0.2, 0.5, 1.0); // Background color (RGBA)
    gl.clear(gl.COLOR_BUFFER_BIT);     // Apply background
  }
</script>
```

3. Web Audio API:

Allows advanced audio processing — generating sounds, applying effects, analyzing audio frequency. Used in music apps and games.

```
<script>
  const audioCtx = new AudioContext();
  const oscillator = audioCtx.createOscillator(); // Sound generator
  oscillator.type = "sine";              // Wave type: sine, square, triangle
  oscillator.frequency.setValueAtTime(440, audioCtx.currentTime); // 440 Hz =
A4
  oscillator.connect(audioCtx.destination); // Connect to speaker
  oscillator.start();                    // Play sound
  oscillator.stop(audioCtx.currentTime + 1); // Stop after 1 second
</script>
```

4. MediaStream API (getUserMedia):

Allows access to camera and microphone. Used in video conferencing and photo capture apps.

```
<video id="vid" autoplay></video>
<script>
  navigator.mediaDevices.getUserMedia({ video: true, audio: false })
    .then(stream => {
      document.getElementById("vid").srcObject = stream;
    })
    .catch(err => console.error('Camera error:', err));
</script>
```

API	Purpose	getContext / Method
Canvas 2D	2D drawing	getContext('2d')
WebGL	3D rendering	getContext('webgl')
Web Audio	Sound synthesis & processing	new AudioContext()
MediaStream	Camera/microphone access	navigator.mediaDevices.getUserMedia()

Q8. What is Tailwind CSS? How is it different from traditional CSS? Discuss the benefits of it.

Tailwind CSS is a utility-first CSS framework that provides a large set of small, single-purpose CSS classes (called utility classes) which you apply directly in your HTML to style elements. Instead of writing custom CSS rules, you compose styles by combining many utility classes like flex, text-center, bg-blue-500, p-4, rounded-lg, etc. Tailwind is developed by Adam Wathan and is widely used in modern web development.

Tailwind CSS vs Traditional CSS:

Aspect	Traditional CSS	Tailwind CSS
Approach	Write custom CSS classes	Apply pre-built utility classes in HTML
CSS File Size	Grows as project grows	Tiny (unused styles purged automatically)
Naming	You must name every class	No naming required
Responsiveness	Write @media queries manually	Prefix classes: md:, lg:, sm:
Consistency	Hard to maintain design system	Built-in design scale (spacing, colors)

Learning Curve	Familiar for beginners	Requires learning utility class names
Customization	Fully custom but verbose	Configured via tailwind.config.js

Example — Traditional CSS:

```
/* styles.css */
.card {
  background-color: white;
  padding: 16px;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
<!-- HTML -->
<div class="card">Hello</div>
```

Example — Tailwind CSS (same result):

```
<div class="bg-white p-4 rounded-lg shadow-md">Hello</div>
```

Benefits of Tailwind CSS:

- Faster Development: Style elements directly in HTML without switching files.
- No CSS Conflicts: Each utility class does one thing — no cascading issues.
- Responsive by Default: Use breakpoint prefixes (sm:, md:, lg:, xl:) for responsive design.
- Dark Mode Support: Use dark: prefix to add dark mode styles easily.
- Smaller Bundle: PurgeCSS removes unused utilities in production.
- Design Consistency: Uses a fixed spacing and color scale — results in a consistent UI.
- Highly Customizable: Extend or override defaults in tailwind.config.js.

Q9. What are the methods to install Tailwind CSS in a project? Explain it.

Tailwind CSS can be installed in multiple ways depending on the type of project. The three main methods are: using npm (Node Package Manager), using the Tailwind CLI, and using a CDN link for quick prototyping. The npm method is recommended for production projects.

Method 1: Install via npm (Recommended)

Step-by-step installation for a Node.js-based project:

1. Initialize your project with npm:

```
npm init -y
```

2. Install Tailwind CSS, PostCSS, and Autoprefixer:

```
npm install -D tailwindcss postcss autoprefixer
```

3. Generate the Tailwind config file:

```
npx tailwindcss init
```

4. Configure the content paths in tailwind.config.js:

```
// tailwind.config.js
module.exports = {
  content: ['./src/**/*.{html,js,jsx,tsx}'],
  theme: { extend: {} },
  plugins: [],
}
```

5. Create the main CSS input file (src/input.css):

```
/* src/input.css */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

6. Build the output CSS:

```
npx tailwindcss -i ./src/input.css -o ./dist/output.css --watch
```

7. Link the output CSS in your HTML:

```
<link href="/dist/output.css" rel="stylesheet">
```

Method 2: Tailwind CSS CDN (Quick Prototyping)

For quick demos, add this script tag to your HTML <head>. Note: Not recommended for production as it loads all utilities.

```
<head>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
```

Method 3: Install with React (Vite or Create React App)

```
npm create vite@latest my-app -- --template react
cd my-app
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Then follow the same config steps as Method 1, and import './index.css' in main.jsx.

Method	Best For	Production Ready
npm + PostCSS	All production projects	Yes
Tailwind CLI	Simple HTML projects	Yes
CDN	Quick demos only	No

Q10. Discuss the significance of layout and background utilities in Tailwind CSS with examples.

Tailwind CSS provides a comprehensive set of utilities for controlling layout and backgrounds, making it easy to design responsive, visually appealing pages without writing custom CSS. These utilities handle everything from display type and positioning to background colors, images, and gradients.

1. Layout Utilities:

Utility Class	CSS Equivalent	Description
block	display: block	Makes element block-level
inline-block	display: inline-block	Inline + block properties
flex	display: flex	Activates flexbox layout
grid	display: grid	Activates grid layout
hidden	display: none	Hides the element
container	max-width: responsive	Centers and constrains content width
relative	position: relative	Relative positioning
absolute	position: absolute	Absolute positioning
fixed	position: fixed	Fixed to viewport
z-10	z-index: 10	Controls stacking order

Layout Example:

```
<div class="container mx-auto px-4">
  <div class="flex items-center justify-between">
    <h1 class="text-2xl font-bold">Logo</h1>
    <nav class="flex gap-4">
      <a class="block px-4 py-2">Home</a>
      <a class="block px-4 py-2">About</a>
    </nav>
  </div>
</div>
```

2. Background Utilities:

Utility Class	Description	Example
bg-blue-500	Solid background color	Blue background
bg-transparent	Transparent background	No background
bg-gradient-to-r	Gradient direction right	Left to right gradient

from-blue-500	Gradient start color	Starts with blue
to-green-500	Gradient end color	Ends with green
bg-cover	Background image covers element	Full cover image
bg-center	Centers background image	Centered image position
bg-no-repeat	No image repetition	Image shows once
bg-fixed	Fixed background on scroll	Parallax effect

Background Example:

```
<!-- Solid color background -->
<div class="bg-blue-500 text-white p-6 rounded-lg">
  Solid blue background
</div>

<!-- Gradient background -->
<div class="bg-gradient-to-r from-purple-500 to-pink-500 text-white p-8">
  Purple to pink gradient
</div>

<!-- Background image -->
<div class="bg-cover bg-center h-64" style="background-image: url('hero.jpg')">
  Hero section with image background
</div>
```

Q11. Describe the use of Flexbox in Tailwind CSS.

Flexbox is a CSS layout model that distributes space and aligns items in a container along a single axis (horizontal or vertical). Tailwind CSS makes Flexbox extremely easy to use with simple utility classes. You just add the flex class to the parent container and then use other flex utilities to control the layout of child elements.

Key Flexbox Utilities in Tailwind:

Utility	CSS Property	Description
flex	display: flex	Enables flexbox on container
flex-row	flex-direction: row	Items arranged horizontally (default)
flex-col	flex-direction: column	Items arranged vertically
flex-wrap	flex-wrap: wrap	Items wrap to next line

flex-nowrap	flex-wrap: nowrap	Items stay in one line
items-start	align-items: flex-start	Align items to top
items-center	align-items: center	Align items to center vertically
items-end	align-items: flex-end	Align items to bottom
justify-start	justify-content: flex-start	Items start from left
justify-center	justify-content: center	Items centered horizontally
justify-between	justify-content: space-between	Equal space between items
justify-around	justify-content: space-around	Equal space around items
gap-4	gap: 1rem	Space between flex items
flex-1	flex: 1 1 0%	Item grows to fill available space
flex-none	flex: none	Item does not grow or shrink

Example 1 — Horizontal Navigation Bar:

```
<nav class="flex items-center justify-between bg-gray-800 text-white px-6 py-
4">
  <h1 class="text-xl font-bold">MyApp</h1>
  <div class="flex gap-6">
    <a href="#" class="hover:text-blue-300">Home</a>
    <a href="#" class="hover:text-blue-300">About</a>
    <a href="#" class="hover:text-blue-300">Contact</a>
  </div>
</nav>
```

Example 2 — Card Layout with Flexbox:

```
<div class="flex flex-wrap gap-4 justify-center p-8">
  <div class="flex flex-col items-center bg-white shadow-md p-6 rounded-lg w-
48">
    <img src="icon.png" class="w-16 h-16 mb-4" alt="icon">
    <h3 class="font-bold text-lg">Feature 1</h3>
    <p class="text-gray-600 text-sm text-center">Description here</p>
  </div>
</div>
```

Example 3 — Centered Content:

```
<div class="flex items-center justify-center h-screen bg-gray-100">
  <div class="bg-white p-8 rounded-xl shadow-lg">
    <h2 class="text-2xl font-bold text-center">Welcome!</h2>
  </div>
</div>
```

Q12. Demonstrate the use of spacing, sizing, and alignment utilities in Tailwind CSS with use cases.

Tailwind CSS provides a comprehensive set of utility classes for spacing (margin and padding), sizing (width and height), and alignment. These follow a consistent numbering scale where the number multiplied by 4px gives the actual size (e.g., p-4 = 16px padding, m-2 = 8px margin).

1. Spacing Utilities (Padding & Margin):

Class	CSS	Value
p-4	padding: 1rem	16px all sides
px-4	padding-left/right: 1rem	16px horizontal
py-4	padding-top/bottom: 1rem	16px vertical
pt-2 / pb-2	padding-top/bottom: 0.5rem	8px top or bottom
m-4	margin: 1rem	16px all sides
mx-auto	margin-left/right: auto	Centers block element
mt-8	margin-top: 2rem	32px top margin
space-x-4	margin between x children	16px gap between horizontal items
space-y-4	margin between y children	16px gap between vertical items

2. Sizing Utilities (Width & Height):

Class	CSS	Description
w-full	width: 100%	Full width of parent
w-1/2	width: 50%	Half width
w-48	width: 12rem	192px fixed width
w-screen	width: 100vw	Full viewport width
h-64	height: 16rem	256px fixed height
h-screen	height: 100vh	Full viewport height
min-w-0	min-width: 0	Minimum width zero
max-w-lg	max-width: 32rem	Constrained maximum width

3. Alignment Utilities:

Class	CSS	Description
text-left	text-align: left	Left-aligned text
text-center	text-align: center	Centered text
text-right	text-align: right	Right-aligned text
align-middle	vertical-align: middle	Vertical alignment
mx-auto	margin: 0 auto	Center block element horizontally

Complete Use Case — Profile Card:

```
<div class="max-w-sm mx-auto mt-16 p-6 bg-white rounded-2xl shadow-lg text-center">
  <img src="avatar.jpg" class="w-24 h-24 rounded-full mx-auto mb-4" alt="Profile">
  <h2 class="text-xl font-bold mb-1">Ansh </h2>
  <p class="text-gray-500 text-sm mb-4">Full Stack Developer</p>
  <div class="flex justify-center space-x-4">
    <button class="px-4 py-2 bg-blue-500 text-white rounded-lg">Follow</button>
    <button class="px-4 py-2 border border-gray-300 rounded-lg">Message</button>
  </div>
</div>
```

Q13. Explain how to create animations using Tailwind CSS.

Tailwind CSS includes several built-in animation utilities that allow you to add motion effects to elements without writing any custom CSS keyframes. These animations are production-ready and highly customizable. For custom animations beyond what's built in, you can extend Tailwind's configuration.

Built-in Animation Classes in Tailwind:

Class	Effect	Common Use Case
animate-spin	Continuous 360° rotation	Loading spinners
animate-ping	Scales up and fades out	Notification badges, alerts
animate-pulse	Fades in and out	Skeleton loading screens
animate-bounce	Bounces up and down	Scroll indicators, alerts
animate-none	Removes any animation	Disable animations

1. Loading Spinner (animate-spin):

```
<div class="flex items-center justify-center h-screen">
  <div class="w-12 h-12 border-4 border-blue-500 border-t-transparent rounded-
full animate-spin"></div>
</div>
```

2. Notification Badge (animate-ping):

```
<div class="relative inline-block">
  <button class="bg-blue-500 text-white px-4 py-2
rounded">Notifications</button>
  <span class="absolute top-0 right-0 w-3 h-3 bg-red-500 rounded-full animate-
ping"></span>
</div>
```

3. Skeleton Loading (animate-pulse):

```
<div class="max-w-sm p-4 bg-white rounded-lg shadow">
  <div class="h-4 bg-gray-200 rounded animate-pulse mb-3"></div>
  <div class="h-4 bg-gray-200 rounded animate-pulse mb-3 w-3/4"></div>
  <div class="h-32 bg-gray-200 rounded animate-pulse"></div>
</div>
```

4. Bounce Arrow (animate-bounce):

```
<div class="flex flex-col items-center mt-20">
  <p class="text-gray-600 mb-2">Scroll Down</p>
  <svg class="w-6 h-6 text-blue-500 animate-bounce" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 9l-7 7-7-7" />
  </svg>
</div>
```

5. Custom Animation via tailwind.config.js:

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      keyframes: {
        wiggle: {
          '0%, 100%': { transform: 'rotate(-3deg)' },
          '50%':  { transform: 'rotate(3deg)' },
        }
      },
      animation: {
        wiggle: 'wiggle 1s ease-in-out infinite',
      }
    }
```

```
  }
}
<!-- Use the custom animation -->
<div class="animate-wiggle text-2xl">□</div>
```

Q14. Explain the role of transformations and SVG utilities in creating interactive UI designs.

Tailwind CSS provides transformation utilities that allow you to scale, rotate, translate (move), and skew HTML elements visually without changing the document flow. These, combined with SVG utilities, help create engaging, interactive UI designs with smooth visual effects.

1. Transform Utilities:

Utility	Effect	Example
scale-105	Scale element to 105%	Grow on hover
scale-90	Scale element to 90%	Shrink effect
-rotate-45	Rotate -45 degrees	Diagonal elements
rotate-180	Rotate 180 degrees	Flip icon on toggle
translate-x-4	Move 16px horizontally	Slide-in effects
translate-y-2	Move 8px vertically	Lift on hover
skew-x-6	Skew X axis 6 degrees	Creative banner shapes
origin-top	Set transform origin to top	Fold/unfold animations

Hover Transformation Example — Interactive Card:

```
<div class="transform transition duration-300 hover:scale-105 hover:-translate-
y-2
     bg-white p-6 rounded-xl shadow-lg cursor-pointer">
  <h3 class="text-lg font-bold">Hover Me!</h3>
  <p class="text-gray-500">This card lifts and grows on hover.</p>
</div>
```

2. Transition Utilities (for smooth animations):

Class	Description
transition	Enables transitions on element
transition-all	Transitions all properties
duration-300	Transition takes 300ms
ease-in-out	Easing curve: slow-fast-slow
delay-150	Waits 150ms before starting

3. SVG Utilities in Tailwind:

Tailwind provides utilities to style inline SVGs using standard color and sizing classes. This is perfect for icon-based UIs.

Class	Effect
fill-current	SVG fill uses text color
stroke-current	SVG stroke uses text color
w-6 h-6	Controls SVG size
text-blue-500	Colors the SVG fill/stroke via currentColor

SVG Icon Button Example:

```
<button class="flex items-center gap-2 px-4 py-2 bg-blue-500 text-white
        rounded-lg transform transition hover:scale-105 hover:bg-blue-600">
  <svg class="w-5 h-5 fill-current" viewBox="0 0 20 20">
    <path d="M10 2a8 8 0 100 16A8 8 0 0010 2zm1 11H9V9h2v4zm0-6H9V5h2v2z"/>
  </svg>
  Learn More
</button>
```

Q15. How can you manage spacing and alignment using Tailwind CSS?

Spacing and alignment are among the most common layout requirements in web development. Tailwind CSS provides a rich set of utility classes for padding, margin, gap, and text/flex alignment that follow a consistent spacing scale. Every spacing value is a multiple of 4px (e.g., m-1 = 4px, m-4 = 16px, m-8 = 32px).

Tailwind Spacing Scale:

Tailwind Number	rem value	px equivalent
1	0.25rem	4px
2	0.5rem	8px
4	1rem	16px
6	1.5rem	24px
8	2rem	32px
12	3rem	48px
16	4rem	64px

Padding Management:

```
<!-- All sides -->
<div class="p-6">Padding 24px all sides</div>
<!-- Horizontal only -->
<div class="px-8">Padding 32px left and right</div>
<!-- Vertical only -->
<div class="py-4">Padding 16px top and bottom</div>
<!-- Individual sides: pt, pb, pl, pr -->
<div class="pt-2 pb-4 pl-6 pr-8">Mixed padding</div>
```

Margin Management:

```
<!-- Center a block element -->
<div class="max-w-lg mx-auto">Centered container</div>
<!-- Top margin for separation -->
<p class="mt-8">Paragraph with top spacing</p>
<!-- Negative margin -->
<div class="-mt-4">Overlapping element</div>
```

Gap (for Flex and Grid):

```
<div class="flex gap-4">
  <div class="bg-blue-200 p-4">Item 1</div>
  <div class="bg-blue-200 p-4">Item 2</div>
  <div class="bg-blue-200 p-4">Item 3</div>
</div>
```

Alignment Examples:

```
<!-- Text alignment -->
<p class="text-center">Centered text</p>
<p class="text-right">Right-aligned text</p>
<!-- Flexbox alignment -->
<div class="flex items-center justify-center h-screen">
  <p>Perfectly centered on screen</p>
</div>
```

Q16. Write SVG tag to draw a rectangle, triangle and circle filled with blue color and a red border.

SVG (Scalable Vector Graphics) allows us to draw shapes directly in HTML using XML-like tags. Below is a complete SVG example that draws three shapes — a rectangle, a triangle (using polygon), and a circle — all filled with blue color and having a red border (stroke).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>SVG Shapes Demo</title>
</head>
<body>
  <h2>SVG Shapes: Rectangle, Triangle, and Circle</h2>

  <svg width="600" height="300" xmlns="http://www.w3.org/2000/svg">

    <!-- Rectangle: x, y = top-left position; width and height of shape -->
    <rect
      x="20" y="80"
      width="150" height="100"
      fill="blue"
      stroke="red"
      stroke-width="4"
    />

    <!-- Triangle: drawn as a polygon with 3 points -->
    <!-- Points format: x1,y1 x2,y2 x3,y3 -->
    <polygon
      points="260,80 200,200 320,200"
      fill="blue"
      stroke="red"
      stroke-width="4"
    />

    <!-- Circle: cx,cy = center point; r = radius -->
```

```
    <circle
      cx="480" cy="150"
      r="80"
      fill="blue"
      stroke="red"
      stroke-width="4"
    />

    <!-- Labels for clarity -->
    <text x="55"  y="220" font-size="14" fill="black">Rectangle</text>
    <text x="225" y="230" font-size="14" fill="black">Triangle</text>
    <text x="450" y="270" font-size="14" fill="black">Circle</text>

  </svg>
</body>
</html>
```

SVG Attribute Explanation:

Attribute	Description
fill	Interior color of the shape (blue)
stroke	Border/outline color (red)
stroke-width	Thickness of the border in pixels
x, y	Top-left position for rect
cx, cy	Center X and Y for circle
r	Radius of circle
points	List of x,y coordinates for polygon vertices

Q17. Discuss the new enhancements in HTML5 forms.

HTML5 brought significant enhancements to web forms, making them more powerful, user-friendly, and functional without requiring JavaScript. These improvements include new input types, new attributes, built-in validation, and new form elements. Together, they allow developers to create richer forms faster with less code.

1. New Input Types:

Input Type	Description	Benefit
email	Validates email format	No JS needed for email check
url	Validates URL format	Auto validation
number	Numeric keyboard + min/max	Built-in range restriction
range	Slider input	Visual value selection
date	Calendar date picker	No JS datepicker needed
time	Time picker	Standardized time input
datetime-local	Date + time together	Single input for both
month	Month and year picker	Billing forms
week	Week and year picker	Schedule forms
color	Color palette picker	Style customization tools
search	Search field with clear button	Better UX
tel	Phone number input	Mobile numeric keyboard

2. New Attributes:

Attribute	Description
placeholder	Ghost text shown in empty field
required	Prevents submission if field is empty
autofocus	Automatically focuses field on page load
autocomplete	on/off — enables browser autocomplete
pattern	Regex pattern for custom validation
min / max / step	For numeric and date ranges
multiple	Allows multiple values or files
form	Links input to any form by its ID
novalidate	Disables HTML5 validation on the form

list	Links to a <datalist> for suggestions

3. New Form Elements:

- <datalist> — Provides autocomplete suggestion list for an input.
- <output> — Displays the result of a calculation.
- <meter> — Represents a scalar value within a range (e.g., battery level).
- <progress> — Shows progress of a task.

Complete Enhanced Form Example:

```
<form action="/register" method="POST" novalidate>
  <input type="text" placeholder="Full Name" required autofocus>
  <input type="email" placeholder="Email" required autocomplete="email">
  <input type="password" minlength="8" placeholder="Password (min 8 chars)">
  <input type="number" min="18" max="100" placeholder="Age">
  <input type="date" name="dob">
  <input type="range" min="0" max="100" step="10">
  <input type="color" name="favcolor">
  <input type="tel" pattern="[0-9]{10}" placeholder="Phone (10 digits)">
  <input list="browsers" placeholder="Choose a browser">
  <datalist id="browsers">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Safari">
  </datalist>
  <progress value="70" max="100">70%</progress>
  <button type="submit">Register</button>
</form>
```

Q18. How can you create a responsive grid layout using Tailwind CSS?

A responsive grid layout adapts the number of columns based on the screen size (mobile, tablet, desktop). Tailwind CSS makes this extremely easy using its grid utilities combined with responsive breakpoint prefixes. The grid system in Tailwind is based on CSS Grid.

Core Grid Utilities:

Class	CSS Property	Description
grid	display: grid	Activates CSS Grid on the container
grid-cols-1	grid-template-columns: repeat(1, ...)	1 column layout
grid-cols-2	grid-template-columns: repeat(2, ...)	2 column layout
grid-cols-3	grid-template-columns: repeat(3, ...)	3 column layout
grid-cols-4	grid-template-columns: repeat(4, ...)	4 column layout
gap-4	gap: 1rem	Space between grid items
col-span-2	grid-column: span 2	Item spans 2 columns
row-span-2	grid-row: span 2	Item spans 2 rows

Responsive Breakpoints in Tailwind:

Prefix	Screen Size	Min Width
(none)	Mobile first (all screens)	0px
sm:	Small screens	640px
md:	Medium screens (tablets)	768px
lg:	Large screens (laptops)	1024px
xl:	Extra large screens	1280px

Example 1 — Responsive Card Grid (1 → 2 → 3 columns):

```
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6 p-8">
  <div class="bg-white p-6 rounded-xl shadow-md">
    <h3 class="font-bold text-lg mb-2">Card 1</h3>
    <p class="text-gray-600">Description of card one.</p>
  </div>
  <div class="bg-white p-6 rounded-xl shadow-md">
```

```
    <h3 class="font-bold text-lg mb-2">Card 2</h3>
    <p class="text-gray-600">Description of card two.</p>
  </div>
  <div class="bg-white p-6 rounded-xl shadow-md">
    <h3 class="font-bold text-lg mb-2">Card 3</h3>
    <p class="text-gray-600">Description of card three.</p>
  </div>
</div>
```

Example 2 — Dashboard Layout (Sidebar + Main + Widget):

```
<div class="grid grid-cols-1 md:grid-cols-4 gap-4 p-6">
  <!-- Sidebar: spans 1 col on all, 1 of 4 on md+ -->
  <div class="md:col-span-1 bg-gray-800 text-white p-4 rounded-
lg">Sidebar</div>

  <!-- Main content: spans full on mobile, 2 cols on md+ -->
  <div class="md:col-span-2 bg-white p-6 rounded-lg shadow">Main Content</div>

  <!-- Widget: spans full on mobile, 1 col on md+ -->
  <div class="md:col-span-1 bg-blue-50 p-4 rounded-lg shadow">Widget</div>
</div>
```

This approach ensures the layout is mobile-first — it starts with 1 column on small screens and expands to multiple columns on larger screens using breakpoint prefixes.

UNIT 2: Introduction to React JS

Q19. What are the key features of React JS?

React JS is an open-source JavaScript library developed by Facebook (now Meta) in 2013 for building fast, interactive, and scalable user interfaces. It is component-based, meaning the UI is broken down into small, reusable pieces. React is widely used for building Single Page Applications (SPAs) and is one of the most popular frontend libraries.

Key Features of React JS:

Feature	Description
Component-Based Architecture	UI is built from small, reusable, independent components
Virtual DOM	React maintains a lightweight virtual copy of the real DOM for performance
JSX (JavaScript XML)	Syntax extension allowing HTML-like code inside JavaScript
One-Way Data Flow	Data flows from parent to child via props — predictable state management
Hooks	Functions like useState, useEffect enable state and lifecycle in functional components
React Router	Client-side routing for SPAs without page reloads
Context API	Share state globally without prop drilling
Server-Side Rendering	React can render on server (Next.js) for better SEO and performance
Rich Ecosystem	Works with Redux, Axios, Material UI, Tailwind CSS, etc.
Developer Tools	React DevTools browser extension for debugging

How Virtual DOM Works:

1. When state changes, React creates a new Virtual DOM tree.
2. It compares (diffs) the new tree with the previous one.
3. Only the elements that changed are updated in the real DOM.
4. This makes updates extremely fast compared to direct DOM manipulation.

Simple React Component Example:

```
import React from 'react';

// Functional component — a reusable UI building block
function Welcome({ name }) {
  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>Welcome to React JS.</p>
    </div>
  );
}

export default Welcome;
```

Q20. Describe the process of setting up a React environment.

Setting up a React development environment requires Node.js and npm (Node Package Manager) installed on your system. There are two main ways to set up React: using Create React App (CRA) for a traditional setup, or using Vite for a faster, modern build setup. Vite is now the preferred choice in 2024-25.

Prerequisites:

- Node.js (version 18 or above) — download from nodejs.org
- npm (comes with Node.js) or yarn
- A code editor — VS Code is recommended

Method 1: Create React App (CRA)

1. Check Node.js version:

```
node -v    // Should show v18+
npm -v     // Should show v9+
```

2. Create a new React project:

```
npx create-react-app my-app
```

3. Navigate to project folder:

```
cd my-app
```

4. Start the development server:

```
npm start
```

App opens at http://localhost:3000 in the browser.

Method 2: Vite (Recommended — Much Faster)

1. Create a Vite + React project:

```
npm create vite@latest my-react-app -- --template react
```

2. Navigate to folder and install dependencies:

```
cd my-react-app
npm install
```

3. Start the dev server:

```
npm run dev
```

App runs at http://localhost:5173

React Project Structure (Vite):

File / Folder	Purpose
src/	All source code — components, pages, styles
src/main.jsx	Entry point — renders App into root div
src/App.jsx	Root component
public/	Static assets (images, favicon)
index.html	HTML template where React mounts
package.json	Project dependencies and scripts
vite.config.js	Vite build configuration

src/main.jsx — Entry Point:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Q21. What is a component in React JS? Explain different types of components with example.

A component in React is an independent, reusable piece of UI — like a function that returns HTML (via JSX). You build complex UIs by combining many small components together. Think of a component like a LEGO brick — each brick is independent, and you can combine them to build something larger.

Types of React Components:

Type	Definition	State Support	Lifecycle
Functional Component	A plain JavaScript function returning JSX	Yes (via Hooks)	Yes (via Hooks)
Class Component	ES6 class extending React.Component	Yes (this.state)	Yes (lifecycle methods)

1. Functional Component (Modern — Recommended):

```
import React from 'react';

// Simple functional component — no state
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}

export default Greeting;
```

2. Class Component (Legacy — Older Style):

```
import React, { Component } from 'react';

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };  // State initialization
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  }

  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increment}>Increment</button>
      </div>
    );
```

```
  }
}

export default Counter;
```

3. Other Component Categories:

- Container Components: Handle logic, state, and data fetching. Pass data to Presentational components.
- Presentational Components: Only receive props and render UI — no state or logic.
- Higher-Order Components (HOC): A function that takes a component and returns an enhanced component.
- Pure Components: React.PureComponent skips re-render if props/state haven't changed.

Q22. Explain functional components in React JS with proper example.

Functional components are JavaScript functions that accept props as input and return JSX (UI). They are the modern and recommended way to write React components. Before React 16.8, functional components couldn't have state or lifecycle methods — but Hooks changed that completely. Now functional components can do everything class components can, with much cleaner syntax.

Characteristics of Functional Components:

- Defined as a regular JavaScript function or arrow function.
- Returns JSX describing the UI.
- Receives props as function argument.
- Can use React Hooks (useState, useEffect, etc.) for state and side effects.
- Shorter, cleaner code compared to class components.
- Easier to test and debug.

Example 1 — Basic Functional Component:

```
import React from 'react';

// Arrow function component receiving props
const UserCard = ({ name, age, city }) => {
  return (
    <div style={{ border: "1px solid #ccc", padding: "20px", borderRadius:
"8px" }}>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>City: {city}</p>
    </div>
  );
```

```
};

export default UserCard;
```

Example 2 — Functional Component with useState:

```
import React, { useState } from 'react';

function LikeButton() {
  // useState returns [currentValue, setterFunction]
  const [likes, setLikes] = useState(0);

  const handleClick = () => {
    setLikes(likes + 1);  // Update state — triggers re-render
  };

  return (
    <div>
      <p>Likes: {likes}</p>
      <button onClick={handleClick}>Like □</button>
    </div>
  );
}

export default LikeButton;
```

Example 3 — Using Component in App.jsx:

```
import React from 'react';
import UserCard from './UserCard';
import LikeButton from './LikeButton';

function App() {
  return (
    <div>
      <UserCard name="Ansh " age={21} city="Ahmedabad" />
      <LikeButton />
    </div>
  );
}

export default App;
```

Q23. What is the difference between React Props and States?

Props and State are both plain JavaScript objects used to control data in a React component, but they serve different purposes and have different behaviors. Understanding the difference between them is fundamental to React development.

Aspect	Props	State
Full Form	Properties	State
Definition	Data passed from parent to child	Data managed within a component
Mutability	Read-only — cannot be changed by receiving component	Mutable — changed via setState/setXxx
Ownership	Owned by parent, received by child	Owned by the component itself
Who sets it?	Parent component sets props	Component sets its own state
Re-render trigger	When parent re-renders with new props	When state is updated via setter function
Direction	One-way (parent → child)	Internal to component
Hook	Accessed via function argument (props)	Managed by useState Hook
Default value	Set via defaultProps	Set in useState(initialValue)

Props Example:

```
// Parent passes data via props
function Parent() {
  return <Child message='Hello from Parent' count={10} />;
}

// Child receives props — READ ONLY
function Child({ message, count }) {
  return <p>{message} — Count is: {count}</p>;
}
```

State Example:

```
import { useState } from 'react';

function Toggle() {
  // State: owned by this component, mutable
  const [isOn, setIsOn] = useState(false);
```

```
  return (
    <div>
      <p>Light is: {isOn ? "ON" : "OFF"}</p>
      <button onClick={() => setIsOn(!isOn)}>Toggle</button>
    </div>
  );
}
```

Key Rule: Props go down (parent → child). Events go up (child → parent via callback). State stays local. This is React's unidirectional data flow.

Q24. How can you pass data from a parent component to a child component using props?

In React, props (short for properties) are the mechanism for passing data from a parent component to a child component. Props are passed as HTML attributes in JSX and received as a function argument in the child component. Props are read-only — the child cannot modify them.

Step-by-step Example — Passing Multiple Props:

```
// ── ProductCard.jsx (Child Component) ──
import React from 'react';

// Destructure props from the function parameter
function ProductCard({ name, price, inStock, category }) {
  return (
    <div style={{ border: "1px solid #ddd", padding: "16px", margin: "8px",
borderRadius: "8px" }}>
      <h3>{name}</h3>
      <p>Category: {category}</p>
      <p>Price: ₹{price}</p>
      <p style={{ color: inStock ? "green" : "red" }}>
        {inStock ? 'In Stock' : 'Out of Stock'}
      </p>
    </div>
  );
}

export default ProductCard;

// ── App.jsx (Parent Component) ──
import React from 'react';
import ProductCard from './ProductCard';

function App() {
  // Data to pass as props
  const products = [
```

```
    { id: 1, name: 'Laptop', price: 55000, inStock: true,  category:
'Electronics' },
    { id: 2, name: 'Mouse',  price: 850,   inStock: false, category:
'Accessories' },
    { id: 3, name: 'Keyboard', price: 1200, inStock: true, category:
'Accessories' },
  ];

  return (
    <div style={{ display: "flex", flexWrap: "wrap", padding: "20px" }}>
      <h1>Products</h1>
      {products.map(product => (
        // Spread object as props — each key becomes a prop
        <ProductCard
          key={product.id}
          name={product.name}
          price={product.price}
          inStock={product.inStock}
          category={product.category}
        />
      ))}
    </div>
  );
}

export default App;
```

Types of Data You Can Pass as Props:

Data Type	Example
String	name='Ansh'
Number	age={21}
Boolean	isLoggedIn={true}
Array	items={['a', 'b', 'c']}
Object	user={{ name: 'Ansh', city: 'Ahmedabad' }}
Function	onClick={handleClick}
JSX Element	icon={<img src='icon.png' />}

Q25. What are React Hooks? Explain commonly used React Hooks with proper example.

React Hooks are built-in functions introduced in React 16.8 that allow functional components to use state, lifecycle features, context, and other React capabilities that were previously only available in class components. Hooks make functional components fully featured and replace the need for class components entirely.

Rules of Hooks:

- Only call Hooks at the top level — not inside loops, conditions, or nested functions.
- Only call Hooks from React functional components or custom Hooks — not regular JS functions.

Commonly Used React Hooks:

Hook	Purpose
useState	Add state to a functional component
useEffect	Handle side effects (API calls, timers, DOM updates)
useContext	Access React Context (global state)
useRef	Reference DOM elements or persist values without re-render
useMemo	Memoize expensive calculated values
useCallback	Memoize functions to prevent re-creation on re-render
useReducer	Complex state management (like Redux, locally)

1. useState — State Management:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // Initial value = 0
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
    </div>
  );
}
```

2. useEffect — Side Effects:

```
import { useState, useEffect } from 'react';

function DataFetcher() {
  const [data, setData] = useState(null);

  useEffect(() => {
    // Runs after component mounts
    fetch('https://jsonplaceholder.typicode.com/posts/1')
      .then(res => res.json())
      .then(json => setData(json));

    // Cleanup function (runs on unmount)
    return () => console.log('Component unmounted');
  }, []); // Empty array = run once on mount

  return <div>{data ? data.title : 'Loading...'}</div>;
}
```

3. useRef — DOM Reference:

```
import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null); // Reference to input DOM element

  const handleFocus = () => {
    inputRef.current.focus(); // Programmatically focus the input
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="I will get focused" />
      <button onClick={handleFocus}>Focus</button>
    </div>
  );
}
```

Q26. Explain the useState Hook with an example.

The useState Hook is the most fundamental React Hook. It allows functional components to have local state — a piece of data that, when changed, causes the component to re-render and display the updated value. Without useState, functional components would be stateless (just display static data).

Syntax:

```
const [stateVariable, setterFunction] = useState(initialValue);
```

Parameter and Return Values:

Part	Description
stateVariable	The current value of the state
setterFunction	Function to update the state (triggers re-render)
initialValue	The starting value (any type: number, string, array, object, boolean)

Example 1 — Counter App:

```
import React, { useState } from 'react';

function Counter() {
  // Declare state variable 'count' with initial value 0
  const [count, setCount] = useState(0);

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>Counter App</h1>
      <h2 style={{ fontSize: '3rem' }}>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment +</button>
      <button onClick={() => setCount(count - 1)}>Decrement -</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
export default Counter;
```

Example 2 — Form Input with useState:

```
import React, { useState } from 'react';

function LoginForm() {
  const [email, setEmail]       = useState('');
  const [password, setPassword] = useState('');
  const [message, setMessage]   = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (email && password) {
      setMessage(`Welcome, ${email}!`);
    } else {
      setMessage('Please fill all fields.');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type='email' value={email}
        onChange={e => setEmail(e.target.value)} placeholder='Email' /><br/>
      <input type='password' value={password}
        onChange={e => setPassword(e.target.value)} placeholder='Password'
/><br/>
      <button type='submit'>Login</button>
      <p>{message}</p>
    </form>
  );
}
export default LoginForm;
```

Q27. How does React Router help in building single-page applications?

React Router is a standard library for routing in React. In a Single Page Application (SPA), the browser loads one HTML page and dynamically updates the content as the user navigates — without full page reloads. React Router enables this by intercepting URL changes and rendering the appropriate React component for each URL, while keeping the page in memory.

How React Router Works — The Process:

1. User visits or clicks a link to a URL (e.g., /about).
2. React Router intercepts the URL change using browser History API — no HTTP request is made.
3. Router matches the URL against defined routes.
4. The matched component is rendered in place — only that component's DOM updates.
5. The page doesn't reload — the React app stays alive.

Benefits for SPAs:

Benefit	Description
No Page Reload	Navigation is instant — no full HTTP request cycle
Nested Routes	Child routes render inside parent route layouts
URL Parameters	Dynamic segments in URL (/user/:id) for dynamic content
Programmatic Navigation	Navigate from code using useNavigate() hook
Protected Routes	Redirect unauthenticated users to login page
404 Handling	Define a catch-all route for unmatched URLs
Code Splitting	Load route components lazily for better performance

Basic React Router Setup Example:

```
npm install react-router-dom

// App.jsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import NotFound from './pages/NotFound';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to='/'>Home</Link>
        <Link to='/about'>About</Link>
      </nav>
      <Routes>
        <Route path='/' element={<Home />} />
        <Route path='/about' element={<About />} />
        <Route path='*' element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Q28. Explain concept of Routing in React JS with proper example.

Routing in React refers to the mechanism that determines which UI component to display based on the current URL. React Router DOM is the library that provides this capability. It maps URL paths to React components, enabling navigation between different views of the application without reloading the page.

Core Routing Concepts:

Concept	Description
Route	Maps a URL path to a component
Routes	Container holding all Route definitions
BrowserRouter	Wraps app, enables History API routing
Link	Renders <a> without page reload
NavLink	Like Link but adds active class when URL matches
useNavigate()	Hook to programmatically navigate
useParams()	Hook to read URL parameters (:id)
Nested Routes	Child routes rendered inside parent using <Outlet>

Complete Routing Example:

```
// Install: npm install react-router-dom

// ── pages/Home.jsx ──
export default function Home()    { return <h2>Home Page</h2>; }

// ── pages/About.jsx ──
export default function About()   { return <h2>About Page</h2>; }

// ── pages/Contact.jsx ──
export default function Contact() { return <h2>Contact Page</h2>; }

// ── pages/UserDetail.jsx (Dynamic Route) ──
import { useParams } from 'react-router-dom';
export default function UserDetail() {
  const { id } = useParams(); // Extract :id from URL
  return <h2>User Profile for ID: {id}</h2>;
}

// ── App.jsx (Main Router Setup) ──
import { BrowserRouter, Routes, Route, NavLink, useNavigate } from 'react-
router-dom';
```

```
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';
import UserDetail from './pages/UserDetail';

function Navbar() {
  const navigate = useNavigate();
  return (
    <nav style={{ display: 'flex', gap: '16px', padding: '12px', background:
'#333', color: 'white' }}>
      <NavLink to='/'       style={({ isActive }) => ({ color: isActive ?
'yellow' : 'white' })}>Home</NavLink>
      <NavLink to='/about'  style={({ isActive }) => ({ color: isActive ?
'yellow' : 'white' })}>About</NavLink>
      <NavLink to='/contact' style={({ isActive }) => ({ color: isActive ?
'yellow' : 'white' })}>Contact</NavLink>
      <button onClick={() => navigate('/user/42')}>Go to User 42</button>
    </nav>
  );
}

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path='/'          element={<Home />} />
        <Route path='/about'     element={<About />} />
        <Route path='/contact'   element={<Contact />} />
        <Route path='/user/:id'  element={<UserDetail />} />
        <Route path='*'          element={<h2>404 - Page Not Found</h2>} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Q29. How is form validation handled in React JS? Explain with proper example.

Form validation in React ensures user input is correct before submission. React provides controlled components — where form inputs are bound to state — which makes validation straightforward. Validation can be done on change (real-time), on blur (when user leaves field), or on submit. External libraries like Formik and React Hook Form are also used for complex forms.

Types of Validation:

Type	When it Runs	Use Case
On Change	Every keystroke	Password strength, live feedback
On Blur	When input loses focus	Email format check
On Submit	When form is submitted	Final check before API call

Complete Form Validation Example:

```
import React, { useState } from 'react';

function RegistrationForm() {
  const [form, setForm] = useState({ name: '', email: '', password: '' });
  const [errors, setErrors] = useState({});
  const [success, setSuccess] = useState('');

  // Update form field on change
  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  // Validate all fields — returns errors object
  const validate = () => {
    const newErrors = {};
    if (!form.name.trim())
      newErrors.name = 'Name is required.';
    if (!form.email.match(/^[^\s@]+@[^\s@]+\.[^\s@]+$/))
      newErrors.email = 'Enter a valid email address.';
    if (form.password.length < 8)
      newErrors.password = 'Password must be at least 8 characters.';
    return newErrors;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const validationErrors = validate();
    if (Object.keys(validationErrors).length > 0) {
      setErrors(validationErrors); // Show errors
```

```
      setSuccess('');
    } else {
      setErrors({});
      setSuccess('Registration successful!');
    }
  };

  return (
    <form onSubmit={handleSubmit} style={{ maxWidth: '400px', margin: '40px
auto' }}>
      <h2>Register</h2>
      <div>
        <input name='name' value={form.name} onChange={handleChange}
               placeholder='Full Name' style={{ width: '100%', padding: '8px',
marginBottom: '4px' }} />
        {errors.name && <p style={{ color: 'red', fontSize: '12px'
}}>{errors.name}</p>}
      </div>
      <div>
        <input name='email' value={form.email} onChange={handleChange}
               placeholder='Email Address' style={{ width: '100%', padding:
'8px', marginBottom: '4px' }} />
        {errors.email && <p style={{ color: 'red', fontSize: '12px'
}}>{errors.email}</p>}
      </div>
      <div>
        <input name='password' type='password' value={form.password}
onChange={handleChange}
               placeholder='Password (min 8 chars)' style={{ width: '100%',
padding: '8px', marginBottom: '4px' }} />
        {errors.password && <p style={{ color: 'red', fontSize: '12px'
}}>{errors.password}</p>}
      </div>
      <button type='submit' style={{ padding: '10px 24px', background:
'#3b82f6', color: 'white', border: 'none', borderRadius: '4px' }}>
        Register
      </button>
      {success && <p style={{ color: 'green', marginTop: '12px'
}}>{success}</p>}
    </form>
  );
}
export default RegistrationForm;
```

Q30. Describe the role of the <BrowserRouter>, <Routes> and <Route> components in React Router.

React Router DOM is built around three core components that work together to enable client-side routing: BrowserRouter, Routes, and Route. Each plays a specific role in the routing system.

1. <BrowserRouter>:

BrowserRouter is the top-level routing container. It uses the HTML5 History API (pushState, popState) to keep the UI in sync with the browser URL. It must wrap all routing components in your application.

- Provides the routing context to all child components.
- Listens to browser URL changes using the History API.
- Does not render any visible UI — it's a logical wrapper.
- All Link, NavLink, Route components must be inside BrowserRouter.

2. <Routes>:

Routes is the container for all Route definitions. It replaced the old Switch component in React Router v6. Routes looks at the current URL and renders only the first Route that matches.

- Always renders only ONE matching route at a time (exclusive matching).
- Automatically picks the best match — no need to write 'exact' keyword.
- Should wrap all your Route components.

3. <Route>:

Route is the core building block. It defines the mapping between a URL path and the React component to render. When the URL matches the path, the given element is rendered.

Prop	Description
path	The URL path to match (e.g., '/about', '/user/:id')
element	The JSX element/component to render on match
index	Marks this as the index (default) child route

Complete Example Showing All Three Together:

```
import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// Page components
const Home    = () => <h1>Home Page</h1>;
const About   = () => <h1>About Page</h1>;
const Product = () => <h1>Products Page</h1>;
const NotFound = () => <h1>404 - Not Found</h1>;
```

```
function App() {
  return (
    // 1. BrowserRouter — root wrapper for all routing
    <BrowserRouter>
      <nav>
        <Link to='/'>Home</Link>  |
        <Link to='/about'>About</Link>  |
        <Link to='/products'>Products</Link>
      </nav>

      {/* 2. Routes — holds all Route definitions */}
      <Routes>
        {/* 3. Route — maps path to component */}
        <Route path='/'         element={<Home />} />
        <Route path='/about'    element={<About />} />
        <Route path='/products' element={<Product />} />
        <Route path='*'         element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Q31. How does React Router manage navigation between different pages in a single-page application (SPA)?

In a traditional multi-page website, clicking a link triggers a full page reload — the browser sends an HTTP request for a new HTML file. In a React SPA, React Router manages navigation entirely in JavaScript without page reloads. It intercepts navigation events and renders the appropriate component based on the URL.

How React Router Manages Navigation:

Navigation Method	Component/Hook	Description
Declarative Navigation	<Link>	Renders anchor tag that doesn't reload page
Active Link Styling	<NavLink>	Adds active class/style to current route link
Programmatic Navigation	useNavigate()	Navigate from code (e.g., after form submit)
URL Parameters	useParams()	Read dynamic segments from URL (:id)
Query Strings	useSearchParams()	Read ?key=value from URL
Redirect	useNavigate() or <Navigate>	Redirect user to another route

Example — Complete Navigation System:

```
import { BrowserRouter, Routes, Route, Link, NavLink,
         useNavigate, useParams, Navigate } from 'react-router-dom';
import { useState } from 'react';

// Protected Route: redirects if not logged in
function ProtectedRoute({ children, isLoggedIn }) {
  return isLoggedIn ? children : <Navigate to='/login' />;
}

// Dynamic user page
function UserPage() {
  const { id } = useParams();
  return <h2>Profile for User ID: {id}</h2>;
}

// Dashboard with programmatic navigate
function Dashboard() {
  const navigate = useNavigate();
  return (
    <div>
      <h2>Dashboard</h2>
      <button onClick={() => navigate('/user/101')}>View User 101</button>
      <button onClick={() => navigate(-1)}>Go Back</button>
    </div>
  );
}

function App() {
  const [isLoggedIn] = useState(true);
  return (
    <BrowserRouter>
      <nav>
        <NavLink to='/' end style={({isActive}) => ({ fontWeight: isActive ?
'bold' : 'normal' })}>Home</NavLink>
        <NavLink to='/dashboard' style={({isActive}) => ({ fontWeight: isActive
? 'bold' : 'normal' })}>Dashboard</NavLink>
      </nav>
      <Routes>
        <Route path='/' element={<h1>Home</h1>} />
        <Route path='/dashboard' element={<ProtectedRoute
isLoggedIn={isLoggedIn}><Dashboard /></ProtectedRoute>} />
        <Route path='/user/:id' element={<UserPage />} />
        <Route path='/login' element={<h2>Please Log In</h2>} />
      </Routes>
    </BrowserRouter>
  );
```

```
}
export default App;
```

Q32. Write a React code that connects with PHP and retrieves data using useEffect.

React (frontend) and PHP (backend) can work together in a client-server architecture. React uses the fetch API (or Axios) inside the useEffect Hook to send HTTP requests to a PHP backend, which responds with JSON data. This is a common pattern for building full-stack applications where React handles the UI and PHP handles server-side logic and database operations.

Architecture Flow:

Layer	Technology	Role
Frontend	React JS	Makes HTTP GET request, displays data
Backend	PHP	Processes request, queries DB, returns JSON
Communication	HTTP + JSON	Data format exchanged between layers

PHP Backend (api/users.php):

```
<?php
// Allow CORS so React (localhost:5173) can call PHP (localhost:8000)
header('Content-Type: application/json');
header('Access-Control-Allow-Origin: *');

// Sample user data (normally from MySQL or MongoDB)
$users = [
  ['id' => 1, 'name' => 'Ansh ',   'email' => 'ansh@example.com',   'city' =>
'Ahmedabad'],
  ['id' => 2, 'name' => 'Raj Shah',     'email' => 'raj@example.com',
'city' => 'Surat'],
  ['id' => 3, 'name' => 'Priya Sharma', 'email' => 'priya@example.com',
'city' => 'Vadodara'],
];

echo json_encode(['success' => true, 'users' => $users]);
?>
```

React Frontend (UserList.jsx):

```
import React, { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers]     = useState([]);    // Store fetched users
```

```
  const [loading, setLoading] = useState(true);  // Loading state
  const [error, setError]     = useState(null);  // Error state

  useEffect(() => {
    // Fetch data from PHP backend
    fetch('http://localhost:8000/api/users.php')
      .then(response => {
        if (!response.ok) throw new Error('Network error');
        return response.json();
      })
      .then(data => {
        setUsers(data.users);   // Save users in state
        setLoading(false);       // Done loading
      })
      .catch(err => {
        setError(err.message);  // Store error message
        setLoading(false);
      });
  }, []); // Empty dependency array — runs only once on mount

  if (loading) return <p>Loading users...</p>;
  if (error)   return <p style={{ color: 'red' }}>Error: {error}</p>;

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial' }}>
      <h2>Users from PHP Backend</h2>
      <table border='1' cellPadding='10' style={{ borderCollapse: 'collapse',
width: '100%' }}>
        <thead>
          <tr style={{ background: '#f0f0f0' }}>
            <th>ID</th><th>Name</th><th>Email</th><th>City</th>
          </tr>
        </thead>
        <tbody>
          {users.map(user => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default UserList;
```

Q33. How React.js and PHP Work Together? Explain with proper example.

React.js and PHP can be combined to build full-stack web applications. React serves as the frontend (View layer) — handling UI, user interactions, and state. PHP serves as the backend — handling server-side logic, database queries, authentication, and returning data as JSON through REST APIs. They communicate via HTTP requests (GET, POST, PUT, DELETE).

Architecture Overview:

Layer	Technology	Responsibilities
Frontend (Client)	React JS	UI components, routing, state management, API calls
Backend (Server)	PHP 8	REST API endpoints, business logic, data processing
Database	MySQL / MongoDB	Data persistence
Protocol	HTTP + JSON	Communication format

Example — Full CRUD: Add and View Students:

PHP Backend (api/students.php):

```php
<?php
header('Content-Type: application/json');
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');

// In-memory simulation (use MySQL in production)
$students = [
  ['id' => 1, 'name' => 'Ansh', 'course' => 'CSE'],
  ['id' => 2, 'name' => 'Raj',  'course' => 'IT'],
];

$method = $_SERVER['REQUEST_METHOD'];

if ($method === 'GET') {
  echo json_encode(['students' => $students]);
} elseif ($method === 'POST') {
  $body = json_decode(file_get_contents('php://input'), true);
  $newStudent = [
    'id'     => count($students) + 1,
    'name'   => $body['name'],
    'course' => $body['course'],
```

```
  ];
  $students[] = $newStudent;
  echo json_encode(['success' => true, 'student' => $newStudent]);
}
?>
```

React Frontend (StudentApp.jsx):

```
import React, { useState, useEffect } from 'react';
const API = 'http://localhost:8000/api/students.php';

function StudentApp() {
  const [students, setStudents] = useState([]);
  const [form, setForm]         = useState({ name: '', course: '' });

  // Fetch all students on load
  useEffect(() => {
    fetch(API).then(r => r.json()).then(d => setStudents(d.students));
  }, []);

  // Add new student via POST
  const addStudent = () => {
    fetch(API, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(form),
    })
    .then(r => r.json())
    .then(d => {
      setStudents([...students, d.student]); // Append new student
      setForm({ name: '', course: '' });       // Reset form
    });
  };

  return (
    <div style={{ padding: '24px', maxWidth: '600px', margin: 'auto' }}>
      <h2>Student Manager (React + PHP)</h2>
      <input placeholder='Name'   value={form.name}   onChange={e =>
setForm({...form, name: e.target.value})} />
      <input placeholder='Course' value={form.course} onChange={e =>
setForm({...form, course: e.target.value})} />
      <button onClick={addStudent}>Add Student</button>
      <ul>
        {students.map(s => <li key={s.id}>{s.name} — {s.course}</li>)}
      </ul>
    </div>
  );
}
export default StudentApp;
```

UNIT 3: Introduction to PHP 8

Q34. Explain the new features introduced in PHP 8.

PHP 8 was released in November 2020 and brought the most significant changes to the PHP language in years. It introduced new syntax features, performance improvements through JIT compilation, stronger type safety, and many quality-of-life improvements for developers. PHP 8.1 and 8.2 further added enums, fibers, and readonly properties.

Key New Features in PHP 8:

Feature	Description
JIT Compilation	Just-In-Time compiler for significantly improved CPU-intensive performance
Named Arguments	Pass arguments by parameter name, in any order
Union Types	A parameter can accept multiple types: int|string
Nullsafe Operator	?-> chains null-safe method calls without if-null checks
Match Expression	Improved switch: strict, expression-based, no fall-through
Constructor Promotion	Declare and initialize class properties directly in constructor
Attributes	Native PHP annotations using #[Attribute] syntax
str_contains() / str_starts_with() / str_ends_with()	New string helper functions
Fibers (PHP 8.1)	Cooperative multitasking / coroutine support
Enums (PHP 8.1)	Native enumeration type
Readonly Properties (PHP 8.1)	Properties that can be written only once
Intersection Types (PHP 8.1)	Parameter must satisfy multiple interfaces: A&B

Quick Code Examples:

```
<?php
// Named Arguments
echo str_pad(string: 'hello', length: 10, pad_string: '*', pad_type:
STR_PAD_BOTH);

// Union Types
function formatId(int|string $id): string { return "ID: $id"; }
```

```
// Match Expression
$status = 2;
$label = match($status) {
  1 => 'Active',
  2 => 'Pending',
  3 => 'Inactive',
  default => 'Unknown'
};
echo $label; // Output: Pending

// Nullsafe Operator
$city = $user?->getAddress()?->getCity();

// str_contains
echo str_contains('Hello PHP 8', 'PHP') ? 'Found' : 'Not Found';
?>
```

Q35. What is Just-In-Time (JIT) compilation in PHP 8?

JIT (Just-In-Time) compilation is one of the most significant performance features introduced in PHP 8. Before JIT, PHP used an interpreter that compiled PHP code to opcodes (intermediate bytecode) via OPcache, and then executed those opcodes. With JIT, PHP goes one step further — it compiles the opcodes to native machine code at runtime, allowing the CPU to execute PHP directly without intermediate interpretation.

How PHP Executes Code — Before and After JIT:

Step	Without JIT	With JIT
1	PHP source code	PHP source code
2	Parsed to AST (Abstract Syntax Tree)	Parsed to AST
3	Compiled to opcodes (stored in OPcache)	Compiled to opcodes (stored in OPcache)
4	Opcodes interpreted by Zend VM	JIT compiles hot opcodes to native CPU machine code
5	CPU executes via Zend VM overhead	CPU executes native code directly — much faster

Types of JIT in PHP 8:

- Tracing JIT: Traces the execution path of frequently run code and compiles it. Best for mathematical and CPU-intensive workloads.
- Function JIT: Compiles entire functions to native code.

JIT Configuration in php.ini:

```
; Enable OPcache (required for JIT)
opcache.enable=1
opcache.memory_consumption=256

; Enable JIT
opcache.jit_buffer_size=100M
opcache.jit=tracing   ; or 'function'
```

Performance Impact:

Workload Type	Performance Gain from JIT
CPU-intensive code (math, algorithms)	2x to 3x faster
Typical web requests (I/O bound)	Minimal improvement (~10%)
Scientific computing / image processing	Significant improvement

Limitations: JIT mainly helps CPU-bound tasks. Most PHP web applications are I/O bound (waiting for database/network), so JIT doesn't drastically improve typical web performance. However, it opens PHP to new domains like machine learning and game scripting.

Q36. Explain the purpose of the Nullsafe operator in PHP 8.

The Nullsafe Operator (?->) was introduced in PHP 8.0 to handle null values gracefully in method and property chains. Before PHP 8, accessing a method or property on a potentially null object required multiple null checks using if statements. The Nullsafe operator short-circuits the chain and returns null if any part of the chain evaluates to null.

Problem Without Nullsafe Operator (PHP 7 style):

```php
<?php
// Old way — verbose and repetitive
$user = getUser(1);

if ($user !== null) {
  $address = $user->getAddress();
  if ($address !== null) {
    $city = $address->getCity();
    if ($city !== null) {
      $name = $city->getName();
    }
  }
}
```

```
echo $name ?? 'Unknown';
```

Solution with Nullsafe Operator (PHP 8):

```
<?php
// Clean, concise — same logic in ONE line
$name = getUser(1)?->getAddress()?->getCity()?->getName();
echo $name ?? 'Unknown';
```

Complete Example with Classes:

```
<?php
class City {
  public function __construct(public string $name) {}
}

class Address {
  public ?City $city;
  public function __construct(?City $city) { $this->city = $city; }
  public function getCity(): ?City { return $this->city; }
}

class User {
  public ?Address $address;
  public function __construct(?Address $address) { $this->address = $address; }
  public function getAddress(): ?Address { return $this->address; }
}

// User with address and city
$user1 = new User(new Address(new City('Ahmedabad')));
echo $user1?->getAddress()?->getCity()?->name; // Output: Ahmedabad

// User with no address (null)
$user2 = new User(null);
echo $user2?->getAddress()?->getCity()?->name ?? 'No city'; // Output: No city
?>
```

The Nullsafe operator is particularly useful in Laravel and other frameworks where model relationships may return null (e.g., $order?->user?->address?->city).

Q37. What are Union Types in PHP 8, and how are they used?

Union Types, introduced in PHP 8.0, allow a function parameter, return type, or property to accept more than one type. Instead of accepting only int or only string, you can declare a parameter as int|string, meaning it accepts either type. PHP validates the type at runtime and throws a TypeError if the wrong type is passed.

Syntax:

```
function functionName(Type1|Type2 $param): ReturnType1|ReturnType2 {
  // ...
}
```

Example 1 — Basic Union Type:

```
<?php
// Function accepts both int and string as ID
function getUserById(int|string $id): string {
  if (is_int($id)) {
    return "Fetching user by numeric ID: $id";
  } else {
    return "Fetching user by username: $id";
  }
}

echo getUserById(42);       // Fetching user by numeric ID: 42
echo getUserById('ansh');   // Fetching user by username: ansh
?>
```

Example 2 — Union Types with null (Nullable):

```
<?php
// null|string is equivalent to ?string
function greet(null|string $name): string {
  return 'Hello, ' . ($name ?? 'Guest') . '!';
}

echo greet('Ansh'); // Hello, Ansh!
echo greet(null);   // Hello, Guest!
?>
```

Example 3 — Union Type for Return Types:

```
<?php
function divide(int $a, int $b): int|float|false {
  if ($b === 0) return false;     // Division by zero
  $result = $a / $b;
  return is_int($result) ? $result : (float)$result;
}
```

```
var_dump(divide(10, 2));  // int(5)
var_dump(divide(10, 3));  // float(3.333...)
var_dump(divide(10, 0));  // bool(false)
?>
```

Union Type	Meaning
int|string	Accept integer or string
int|float	Accept integer or float
string|null	Accept string or null (same as ?string)
int|float|false	Accept int, float, or boolean false
array|Traversable	Accept array or any Traversable object

Q38. What are Named Arguments? Explain with proper example.

Named Arguments is a feature introduced in PHP 8.0 that allows you to pass arguments to a function by specifying the parameter name, rather than relying on the positional order. This makes function calls more readable and allows you to skip optional parameters that have default values.

Syntax:

```
functionName(paramName: value, anotherParam: value);
```

Benefits of Named Arguments:

- Improved readability — code is self-documenting.
- Skip optional parameters — no need to pass null for unused parameters.
- Order independent — pass arguments in any order.
- Works with built-in PHP functions too.

Example 1 — Basic Named Arguments:

```
<?php
function createUser(string $name, int $age = 18, string $role = 'user'): string
{
  return "$name | Age: $age | Role: $role";
}

// Positional (old way)
echo createUser('Ansh', 21, 'admin');
// Output: Ansh | Age: 21 | Role: admin

// Named Arguments — skip 'age', only provide 'role'
```

```
echo createUser(name: 'Raj', role: 'moderator');
// Output: Raj | Age: 18 | Role: moderator

// Named Arguments in any order
echo createUser(role: 'admin', age: 25, name: 'Priya');
// Output: Priya | Age: 25 | Role: admin
?>
```

Example 2 — Named Arguments with Built-in Functions:

```
<?php
// OLD WAY — confusing which argument is which
echo str_pad('PHP', 10, '-', STR_PAD_BOTH);

// NAMED ARGUMENTS — much clearer!
echo str_pad(
  string: 'PHP',
  length: 10,
  pad_string: '-',
  pad_type: STR_PAD_BOTH
);
// Output: ---PHP----

// array_slice with named arguments
$arr = [1, 2, 3, 4, 5];
$result = array_slice(array: $arr, offset: 1, length: 3, preserve_keys: true);
print_r($result); // [2, 3, 4]
?>
```

Q39. Explain Constructor Property Promotion in PHP 8.

Constructor Property Promotion is a syntax shortcut introduced in PHP 8.0 that allows class properties to be declared and initialized directly in the constructor's parameter list. Before PHP 8, you had to declare the property at the class level, then receive it as a constructor parameter, then assign it using $this->property = $param — three separate steps. PHP 8 reduces this to just one line.

Before PHP 8 (verbose):

```php
<?php
class Product {
  // Step 1: Declare properties
  public string  $name;
  public float   $price;
  public int     $stock;
  public string  $category;

  // Step 2: Constructor parameters + Step 3: Assignments
  public function __construct(
    string $name,
    float  $price,
    int    $stock,
    string $category = 'General'
  ) {
    $this->name     = $name;
    $this->price    = $price;
    $this->stock    = $stock;
    $this->category = $category;
  }
}
```

After PHP 8 (with Constructor Property Promotion):

```php
<?php
class Product {
  // ONE LINE per property — declare + receive + assign, all at once!
  public function __construct(
    public string  $name,
    public float   $price,
    public int     $stock,
    public string  $category = 'General'
  ) {} // Constructor body is empty — PHP handles the rest
}

// Usage is exactly the same
$p = new Product(name: 'Laptop', price: 55000.00, stock: 10);
echo $p->name;     // Laptop
echo $p->price;    // 55000
echo $p->category; // General (default)
?>
```

Works with all visibility modifiers:

```
class UserDTO {
  public function __construct(
    public    readonly int    $id,      // readonly — can only be set once
    public    string          $name,
    protected string          $email,
    private   ?string         $phone = null
  ) {}
}
```

PHP Version	Lines of Code (4 properties)	Approach
PHP 7	~16 lines	Declare + parameter + assignment
PHP 8	~6 lines	Constructor Property Promotion

Q40. What is the Match expression in PHP 8, and how does it differ from a switch statement?

The Match expression was introduced in PHP 8.0 as a more powerful and strict alternative to the switch statement. Like switch, it compares a value against multiple arms, but match uses strict comparison (===), returns a value directly, does not fall through, and is an expression (not a statement) — meaning it can be used on the right side of an assignment.

Feature	switch Statement	match Expression
Comparison	Loose (==)	Strict (===)
Fall-through	Yes — needs break to prevent	No — each arm is independent
Returns value	No — it's a statement	Yes — it's an expression
Multiple conditions	case 1: case 2: stacked	1, 2 => value (comma-separated)
Default	default: block	default => value
No match	Silent (does nothing)	Throws UnhandledMatchError
Syntax	Uses : and break	Uses => and ,

switch Statement (Old Way):

```php
<?php
$status = '2';

switch ($status) {
  case 1:
  case '1': // Loose comparison — both int 1 and string '1' match
    echo 'Active';
    break;
  case 2:
    echo 'Pending';
    break;
  default:
    echo 'Unknown';
}
// Output: Pending (because '2' == 2 in loose comparison)
?>
```

match Expression (PHP 8 Way):

```php
<?php
$status = '2';

// Strict comparison: '2' !== 2
$label = match($status) {
  1, '1' => 'Active',      // Multiple values in one arm
  2      => 'Pending',     // int 2
  '2'    => 'Pending (string)', // string '2' — separate match
  3      => 'Inactive',
  default => 'Unknown'
};

echo $label; // Pending (string) — because '2' === '2'
?>
```

match as Expression in Assignment:

```php
<?php
$role = 'admin';

// match returns a value directly
$permissions = match($role) {
  'admin'     => ['read', 'write', 'delete'],
  'editor'    => ['read', 'write'],
  'viewer'    => ['read'],
  default     => []
};
```

```
print_r($permissions); // ['read', 'write', 'delete']
?>
```

Q41. Describe how to build RESTful APIs using PHP 8.

A RESTful API (Representational State Transfer API) is a web service that follows REST architectural principles, using standard HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations and returning data in JSON format. PHP 8 is well-suited for building lightweight REST APIs without a framework.

REST Principles:

HTTP Method	Action	Endpoint Example
GET	Read / retrieve data	GET /api/users or /api/users/1
POST	Create new resource	POST /api/users
PUT	Update entire resource	PUT /api/users/1
PATCH	Partial update	PATCH /api/users/1
DELETE	Delete resource	DELETE /api/users/1

Complete PHP 8 REST API — users.php:

```
<?php
header('Content-Type: application/json');
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');

// Sample data (use MySQL/MongoDB in production)
$users = [
  ['id' => 1, 'name' => 'Ansh', 'email' => 'ansh@example.com'],
  ['id' => 2, 'name' => 'Raj',  'email' => 'raj@example.com'],
];

$method = $_SERVER['REQUEST_METHOD'];
$id     = isset($_GET['id']) ? (int)$_GET['id'] : null;

function jsonResponse(array $data, int $status = 200): void {
  http_response_code($status);
  echo json_encode($data);
  exit;
}

match($method) {
  'GET'    => handleGet($users, $id),
```

```
  'POST'   => handlePost($users),
  'PUT'    => handlePut($users, $id),
  'DELETE' => handleDelete($users, $id),
  'OPTIONS'=> jsonResponse(['status' => 'ok']),
  default  => jsonResponse(['error' => 'Method not allowed'], 405)
};

function handleGet(array $users, ?int $id): void {
  if ($id) {
    $user = array_filter($users, fn($u) => $u['id'] === $id);
    $user ? jsonResponse(array_values($user)[0]) : jsonResponse(['error' =>
'Not found'], 404);
  } else {
    jsonResponse(['users' => $users]);
  }
}

function handlePost(array $users): void {
  $body = json_decode(file_get_contents('php://input'), true);
  if (!isset($body['name'], $body['email'])) {
    jsonResponse(['error' => 'Name and email required'], 400);
  }
  $new = ['id' => count($users)+1, 'name' => $body['name'], 'email' =>
$body['email']];
  jsonResponse(['success' => true, 'user' => $new], 201);
}

function handlePut(array $users, ?int $id): void {
  $body = json_decode(file_get_contents('php://input'), true);
  jsonResponse(['success' => true, 'updated_id' => $id, 'data' => $body]);
}

function handleDelete(array $users, ?int $id): void {
  jsonResponse(['success' => true, 'deleted_id' => $id]);
}
?>
```

Q42. What are the benefits of using GraphQL with PHP 8?

GraphQL is a query language for APIs developed by Facebook as an alternative to REST. Instead of fixed endpoints that return predefined data, GraphQL allows clients to request exactly the data they need in a single query. With PHP 8, GraphQL can be implemented using libraries like webonyx/graphql-php. It pairs well with PHP 8's typed properties, union types, and attributes.

GraphQL vs REST Comparison:

Aspect	REST API	GraphQL
Endpoints	Multiple endpoints (/users, /posts, /comments)	Single endpoint (/graphql)
Over-fetching	Returns all fields even unused ones	Returns only requested fields
Under-fetching	Multiple requests for related data	Single query fetches related data
Versioning	Requires /v1, /v2 endpoints	No versioning — evolve schema
Type System	No built-in type system	Strongly typed schema (SDL)
Real-time	Requires WebSockets separately	Built-in Subscriptions

Benefits of GraphQL with PHP 8:

- Reduced Over-fetching: Client specifies exact fields. E.g., fetch only user name and email, not the entire user object with 20 fields.
- Single Request for Nested Data: Fetch user + posts + comments in one GraphQL query — no N+1 problem.
- Strong Type System: GraphQL schema acts as documentation and contract between frontend and backend.
- PHP 8 Union Types Map Naturally: PHP 8 union types align well with GraphQL union types (interface types).
- PHP 8 Attributes Integration: Use PHP 8 Attributes (#[GraphQLType]) to annotate classes.
- Better Developer Experience: GraphiQL IDE provides auto-complete and schema exploration.

Installation:

```
composer require webonyx/graphql-php
```

Basic PHP 8 + GraphQL Example:

```
<?php
require 'vendor/autoload.php';
use GraphQL\Type\Definition\Type;
use GraphQL\Type\Definition\ObjectType;
use GraphQL\Type\Schema;
use GraphQL\GraphQL;

$userType = new ObjectType([
  'name' => 'User',
  'fields' => [
    'id'    => Type::int(),
    'name'  => Type::string(),
    'email' => Type::string(),
  ]
]);

$queryType = new ObjectType([
  'name' => 'Query',
  'fields' => [
    'user' => [
      'type'    => $userType,
      'args'    => ['id' => Type::int()],
      'resolve' => fn($root, $args) => ['id' => $args['id'], 'name' => 'Ansh',
'email' => 'ansh@test.com']
    ]
  ]
]);

$schema = new Schema(['query' => $queryType]);
$rawInput = json_decode(file_get_contents('php://input'), true);
$result   = GraphQL::executeQuery($schema, $rawInput['query'])->toArray();
echo json_encode($result);
?>
```

Q43. How is JWT authentication implemented in PHP 8?

JWT (JSON Web Token) is a compact, self-contained method for securely transmitting information between parties as a JSON object. It is widely used for stateless authentication in REST APIs — the server issues a signed token to the client after login, and the client sends this token with every subsequent request. The server verifies the token signature without needing a session database.

JWT Structure:

A JWT has three parts separated by dots: Header.Payload.Signature

Part	Content	Example
Header	Algorithm and token type	{"alg":"HS256","typ":"JWT"}
Payload	Claims (user data, expiry)	{"sub":"1","name":"Ansh","exp":1700000000}
Signature	HMAC hash of header+payload	SflKxwRJSMeKKF2QT4fwpMeJf36P...

Installation:

```
composer require firebase/php-jwt
```

JWT Authentication in PHP 8:

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

header('Content-Type: application/json');
header('Access-Control-Allow-Origin: *');

define('JWT_SECRET', 'your_super_secret_key_here_min_32_chars');

// ── Login endpoint: POST /api/login.php ──
function login(): void {
  $body     = json_decode(file_get_contents('php://input'), true);
  $email    = $body['email']    ?? '';
  $password = $body['password'] ?? '';

  // Simulate user lookup (use DB in production)
  if ($email === 'ansh@example.com' && $password === 'secret123') {
    $payload = [
      'iss'  => 'localhost',           // Issuer
      'aud'  => 'localhost',           // Audience
      'iat'  => time(),               // Issued at
      'exp'  => time() + 3600,        // Expires in 1 hour
      'sub'  => 1,                    // User ID
```

```
      'name' => 'Ansh ',
      'role' => 'admin'
    ];
    $token = JWT::encode($payload, JWT_SECRET, 'HS256');
    echo json_encode(['success' => true, 'token' => $token]);
  } else {
    http_response_code(401);
    echo json_encode(['error' => 'Invalid credentials']);
  }
}

// ── Protected endpoint: verifies JWT token ──
function getProtectedData(): void {
  $headers = getallheaders();
  $authHeader = $headers['Authorization'] ?? '';

  if (!str_starts_with($authHeader, 'Bearer ')) {
    http_response_code(401);
    echo json_encode(['error' => 'Token required']);
    return;
  }

  $token = substr($authHeader, 7);
  try {
    $decoded = JWT::decode($token, new Key(JWT_SECRET, 'HS256'));
    echo json_encode(['message' => 'Welcome ' . $decoded->name, 'role' =>
$decoded->role]);
  } catch (\Exception $e) {
    http_response_code(403);
    echo json_encode(['error' => 'Invalid token: ' . $e->getMessage()]);
  }
}

// Route to appropriate function
$path = $_SERVER['REQUEST_URI'];
match(true) {
  str_ends_with($path, '/login')     => login(),
  str_ends_with($path, '/protected') => getProtectedData(),
  default => echo json_encode(['error' => 'Route not found'])
};
?>
```

Q44. Explain the concept of attributes (annotations) in PHP 8.

Attributes (also called Annotations in other languages like Java) are a native way to add structured metadata to classes, methods, properties, parameters, and functions in PHP 8. Before PHP 8, developers used PHPDoc comments (/** @Route('/home') */) to add metadata, but these were not parsed by PHP itself. PHP 8 Attributes are first-class citizens — PHP can read and process them at runtime via the Reflection API.

Syntax:

```
#[AttributeName(arguments)]
```

Where Attributes Can Be Applied:

Target	Example
Class	#[Route('/api')] class ApiController
Method / Function	#[Get('/users')] public function getUsers()
Property	#[Column('user_name')] public string $name
Parameter	function fn(#[Positive] int $count)
Constants	#[Deprecated] const OLD_FLAG = true

Example 1 — Creating and Using a Custom Attribute:

```php
<?php
// Step 1: Define a custom attribute
#[Attribute]
class Route {
  public function __construct(
    public string $path,
    public string $method = 'GET'
  ) {}
}

// Step 2: Apply the attribute to a class and method
#[Route('/api/v1')]
class UserController {

  #[Route(path: '/users', method: 'GET')]
  public function index(): string {
    return json_encode(['users' => []]);
  }

  #[Route(path: '/users', method: 'POST')]
  public function create(): string {
    return json_encode(['message' => 'User created']);
```

```
  }
}

// Step 3: Read attributes using Reflection API
$reflection = new ReflectionClass(UserController::class);

// Read class-level attribute
$classAttr = $reflection->getAttributes(Route::class)[0]->newInstance();
echo "Class path: " . $classAttr->path . PHP_EOL; // /api/v1

// Read method-level attributes
foreach ($reflection->getMethods() as $method) {
  $attrs = $method->getAttributes(Route::class);
  foreach ($attrs as $attr) {
    $route = $attr->newInstance();
    echo "{$method->name}: [{$route->method}] {$route->path}" . PHP_EOL;
  }
}
?>
```

Attributes are heavily used in modern PHP frameworks like Symfony (routing, validation) and Doctrine (ORM column mapping).

Q45. Explain the core principles of Object-Oriented Programming in PHP with examples.

Object-Oriented Programming (OOP) is a programming paradigm that organizes code into objects — instances of classes that bundle data (properties) and behaviour (methods) together. PHP has full OOP support including classes, objects, inheritance, interfaces, abstract classes, and traits. OOP makes code more modular, reusable, and maintainable.

Core OOP Principles:

Principle	Description
Encapsulation	Bundle data and methods; restrict access using visibility (public/private/protected)
Inheritance	Child class inherits properties and methods from parent class
Polymorphism	Different classes respond to the same method name differently
Abstraction	Hide implementation details; expose only necessary interface

1. Encapsulation:

```php
<?php
class BankAccount {
  private float $balance; // private — hidden from outside

  public function __construct(float $initial) {
    $this->balance = $initial;
  }

  // Public getter — controlled access
  public function getBalance(): float { return $this->balance; }

  // Public method — validates before modifying private data
  public function deposit(float $amount): void {
    if ($amount > 0) $this->balance += $amount;
  }

  public function withdraw(float $amount): bool {
    if ($amount > $this->balance) return false;
    $this->balance -= $amount;
    return true;
  }
}

$acc = new BankAccount(5000);
$acc->deposit(2000);
echo $acc->getBalance(); // 7000
?>
```

2. Inheritance:

```php
<?php
class Animal {
  public function __construct(public string $name) {}
  public function eat(): string { return "{$this->name} is eating."; }
}

class Dog extends Animal {
  // Inherited: eat(), $name
  public function bark(): string { return "{$this->name} says Woof!"; }
}

$dog = new Dog('Buddy');
echo $dog->eat();  // Buddy is eating.
echo $dog->bark(); // Buddy says Woof!
?>
```

3. Polymorphism:

```php
<?php
abstract class Shape {
  abstract public function area(): float; // Must be implemented by children
}

class Circle extends Shape {
  public function __construct(private float $radius) {}
  public function area(): float { return M_PI * $this->radius ** 2; }
}

class Rectangle extends Shape {
  public function __construct(private float $w, private float $h) {}
  public function area(): float { return $this->w * $this->h; }
}

$shapes = [new Circle(5), new Rectangle(4, 6)];
foreach ($shapes as $shape) {
  // Same method name area() — different behavior
  echo round($shape->area(), 2) . PHP_EOL;
}
// Output: 78.54 | 24
?>
```

Q46. How do PHP classes and objects work? Illustrate with an example.

A class in PHP is a blueprint or template that defines the structure (properties) and behaviour (methods) that its objects will have. An object is an instance of a class — a concrete entity created from the class blueprint using the new keyword. PHP supports full object-oriented class features including constructors, destructors, static members, magic methods, and interfaces.

Class Definition Structure:

Component	Description
Properties	Variables belonging to the class (data fields)
Constructor (__construct)	Called automatically when object is created
Methods	Functions that define class behaviour
Visibility	public (accessible anywhere), protected (class+children), private (class only)
Static	Belongs to class, not instances — accessed via ClassName::method()
Magic Methods	__toString, __get, __set, __destruct, etc.

Complete Student Class Example:

```php
<?php
class Student {
  // Properties
  private static int $count = 0;  // Tracks total students (class-level)

  // Constructor Property Promotion (PHP 8)
  public function __construct(
    public readonly int    $id,
    public string          $name,
    private string         $email,
    protected float        $cgpa = 0.0
  ) {
    self::$count++; // Increment counter whenever a student is created
  }

  // Method to update CGPA
  public function updateCGPA(float $cgpa): void {
    if ($cgpa >= 0 && $cgpa <= 10) {
      $this->cgpa = $cgpa;
    }
  }

  // Getter for protected property
  public function getCGPA(): float { return $this->cgpa; }

  // Static method — belongs to class, not instance
  public static function getCount(): int { return self::$count; }

  // Magic method — called when object used as string
  public function __toString(): string {
    return "Student[{$this->id}]: {$this->name} | CGPA: {$this->cgpa}";
  }

  // Destructor — called when object is destroyed
  public function __destruct() {
    echo "Student {$this->name} object destroyed." . PHP_EOL;
  }
}

// Creating objects
$s1 = new Student(id: 1, name: 'Ansh ',   email: 'ansh@sou.edu');
$s2 = new Student(id: 2, name: 'Raj Shah',     email: 'raj@sou.edu');
$s3 = new Student(id: 3, name: 'Priya Sharma', email: 'priya@sou.edu');

// Using methods
$s1->updateCGPA(9.2);
```

```
$s2->updateCGPA(8.5);

// Using __toString magic method
echo $s1 . PHP_EOL; // Student[1]: Ansh  | CGPA: 9.2
echo $s2 . PHP_EOL; // Student[2]: Raj Shah | CGPA: 8.5

// Static method call — no instance needed
echo 'Total Students: ' . Student::getCount(); // 3
?>
```

Q47. How can the Stringable interface be used in PHP 8?

The Stringable interface was introduced in PHP 8.0. It represents any class that implements the __toString() magic method. Any class that defines __toString() automatically implements Stringable — you don't need to explicitly declare it (though you can). This interface allows you to type-hint a parameter as Stringable|string, ensuring the accepted value can always be represented as a string.

Why Stringable Was Introduced:

- Before PHP 8, you couldn't type-hint 'any value that can be converted to string'.
- Developers wrote workarounds like accepting mixed or object types.
- Stringable makes this clean and type-safe.

Basic Example:

```
<?php
// PHP automatically adds 'implements Stringable' to any class with __toString
class Money {
  public function __construct(
    private float  $amount,
    private string $currency = 'INR'
  ) {}

  // __toString automatically makes this class Stringable
  public function __toString(): string {
    return "{$this->currency} " . number_format($this->amount, 2);
  }
}

class Temperature {
  public function __construct(private float $celsius) {}
  public function __toString(): string {
    return "{$this->celsius}°C";
  }
}

// Function accepts either a plain string OR any Stringable object
```

```
function printLabel(string|Stringable $value): void {
  echo "Label: {$value}" . PHP_EOL;
}

$price = new Money(1500.50);
$temp  = new Temperature(36.6);

printLabel($price);         // Label: INR 1,500.50
printLabel($temp);          // Label: 36.6°C
printLabel('Free Shipping'); // Label: Free Shipping (plain string)

// Verify interface implementation
var_dump($price instanceof Stringable); // bool(true)
var_dump($temp  instanceof Stringable); // bool(true)
?>
```

Explicitly Implementing Stringable (Optional but Good Practice):

```
<?php
class Address implements Stringable {
  public function __construct(
    private string $street,
    private string $city,
    private string $state
  ) {}

  public function __toString(): string {
    return "{$this->street}, {$this->city}, {$this->state}";
  }
}

$addr = new Address('123 Main St', 'Ahmedabad', 'Gujarat');
echo $addr; // 123 Main St, Ahmedabad, Gujarat
?>
```

Practical Use Case in Logging or Templating:

```
<?php
// Logger that accepts any stringable value
function log(string|Stringable $msg): void {
  $timestamp = date('Y-m-d H:i:s');
  file_put_contents('app.log', "[$timestamp] $msg" . PHP_EOL, FILE_APPEND);
}

log('Application started');
log(new Money(500));  // Works because Money implements Stringable
?>
```

UNIT 4: Database Connectivity Using MongoDB

Q48. What are the key features of MongoDB?

MongoDB is a popular NoSQL, document-oriented database. Instead of storing data in rows and columns like SQL databases, it stores data as JSON-like documents (BSON). It is designed for scalability, flexibility, and high performance.

Analogy:

Think of MongoDB as a filing cabinet where each drawer (collection) holds folders (documents), and each folder can contain different types of papers (fields) — unlike SQL where every folder must have the same structure.

Key Features:

- Document-Oriented Storage: Data is stored as BSON (Binary JSON) documents. Each document is a self-contained unit with key-value pairs.
- Schema-less / Flexible Schema: No fixed schema is required. Different documents in the same collection can have different fields.
- High Performance: Supports indexing, in-memory processing, and efficient query execution for fast reads and writes.
- Horizontal Scalability (Sharding): Data can be distributed across multiple servers (shards) automatically, allowing massive scale.
- Replication (Replica Sets): Data is automatically replicated across multiple servers for high availability and fault tolerance.
- Rich Query Language: Supports filtering, projection, sorting, aggregation, text search, and geospatial queries.
- Aggregation Framework: A powerful pipeline for data transformation and analysis (similar to GROUP BY in SQL).
- Transactions: Supports multi-document ACID transactions (since MongoDB 4.0).
- GridFS: A specification for storing and retrieving large files (images, videos) exceeding the 16MB BSON document limit.
- Ad-hoc Queries: Supports queries on any field without pre-defining queries at design time.

Feature	Description
Document Store	Stores data as JSON-like BSON documents
Schema-less	No rigid table structure required
Sharding	Horizontal scaling across servers
Replica Sets	Automatic data replication for HA
Aggregation	Pipeline-based data processing
Indexing	Fast lookups on any field

Q49. When to use MongoDB?

MongoDB is not a one-size-fits-all solution. It shines in specific scenarios where flexibility, speed, and scale are needed. Choosing the right database depends on the nature of your data and application.

Use MongoDB When:

- Your data is hierarchical or nested: Product catalogs, user profiles, or blog posts with nested comments fit naturally in documents.
- Schema changes frequently: Startups or apps in rapid development where the data model evolves regularly benefit from MongoDB's flexible schema.
- You need high write throughput: Real-time logging, analytics, IoT sensor data, and event tracking where millions of writes happen per second.
- Horizontal scaling is needed: When your data grows beyond what one server can handle, MongoDB's sharding spreads the load automatically.
- You are building real-time applications: Chat apps, live dashboards, and notification systems that need fast reads and writes.
- Working with unstructured or semi-structured data: Content management systems, emails, or social media data where every record looks different.
- Geospatial data: Apps that use location-based features like 'find restaurants near me' leverage MongoDB's built-in geospatial indexing.

Avoid MongoDB When:

- Data requires complex multi-table JOINs: Relational databases like MySQL or PostgreSQL are better suited for highly relational data.
- Strong ACID compliance across many tables: Financial transaction systems where consistency is critical.
- Data is highly structured and uniform: When every record has the same fixed fields, SQL's structure is an advantage.

Scenario	Use MongoDB?
Product catalog with varying attributes	Yes
Real-time chat messages	Yes
Banking transaction ledger	No (use SQL)
IoT sensor data (millions/sec)	Yes
Complex JOIN-heavy reporting	No (use SQL)
Content Management System	Yes

Q50. How do you create a database and collection in MongoDB?

In MongoDB, a database is a container for collections, and a collection is a container for documents. MongoDB uses lazy creation — databases and collections are not physically created until you insert the first document.

Step-by-Step Process:

1. Open your terminal and start the MongoDB shell by typing: mongosh
2. Switch to (or create) a database using the 'use' command.
3. Create a collection explicitly using createCollection(), OR let it auto-create on first insert.
4. Insert a document to trigger physical creation.
5. Verify with show dbs and show collections.

Code Example (MongoDB Shell):

```
// Step 1: Start the shell
mongosh

// Step 2: Create / switch to a database
use studentDB

// Step 3: Explicitly create a collection (optional)
db.createCollection("students")

// Step 4: Insert a document (auto-creates DB and collection)
db.students.insertOne({
  name: "Raj ",
  enrollment: "2302031800037",
  semester: 6,
  branch: "CSE"
})
```

```
// Step 5: Verify
show dbs            // Lists all databases
show collections    // Lists collections in current DB

// Step 6: View the inserted data
db.students.find().pretty()
```

Command	Purpose
use dbName	Switch to or create a database
db.createCollection('name')	Explicitly create a collection
db.collection.insertOne({})	Insert document (auto-creates)
show dbs	List all databases
show collections	List collections in current DB

Note: MongoDB only shows a database in 'show dbs' after at least one document is inserted. An empty database is not listed.

Q51. What are the various methods available in MongoDB for performing CRUD operations? Explain with proper example.

CRUD stands for Create, Read, Update, and Delete — the four fundamental operations on any database. MongoDB provides dedicated methods for each operation on collections.

1. CREATE (Insert):

- insertOne({}) — inserts a single document
- insertMany([{}, {}]) — inserts multiple documents at once

```
// insertOne
db.students.insertOne({
  name: "Priya Shah",
  age: 21,
  course: "FSWD",
  marks: 88
})

// insertMany
db.students.insertMany([
  { name: "Arjun Mehta", age: 22, course: "FSWD", marks: 75 },
  { name: "Sneha Joshi", age: 20, course: "DSA",  marks: 91 }
])
```

2. READ (Find):

- find({}) — retrieves all matching documents
- findOne({}) — retrieves the first matching document
- Projection: Pass a second argument to select specific fields

```
// Find all students
db.students.find({})

// Find with filter
db.students.find({ course: "FSWD" })

// Find one
db.students.findOne({ name: "Priya Shah" })

// Projection (show only name and marks, hide _id)
db.students.find({}, { name: 1, marks: 1, _id: 0 })

// Sort (ascending = 1, descending = -1)
db.students.find().sort({ marks: -1 })

// Limit results
db.students.find().limit(5)
```

3. UPDATE:

- updateOne({filter}, {$set: {}}) — updates the first matching document
- updateMany({filter}, {$set: {}}) — updates all matching documents
- replaceOne({filter}, {newDoc}) — replaces the entire document

```
// Update one document
db.students.updateOne(
  { name: "Priya Shah" },   // filter
  { $set: { marks: 92 } }   // update
)

// Update many documents
db.students.updateMany(
  { course: "FSWD" },
  { $set: { semester: 6 } }
)

// Increment a value
db.students.updateOne(
  { name: "Arjun Mehta" },
  { $inc: { marks: 5 } }  // adds 5 to existing marks
)
```

4. DELETE:

- deleteOne({filter}) — deletes the first matching document
- deleteMany({filter}) — deletes all matching documents

```
// Delete one document
db.students.deleteOne({ name: "Arjun Mehta" })

// Delete with filter
db.students.deleteMany({ marks: { $lt: 40 } })
```

Operation	Method	Purpose
Create	insertOne / insertMany	Add documents
Read	find / findOne	Retrieve documents
Update	updateOne / updateMany	Modify documents
Delete	deleteOne / deleteMany	Remove documents

Q52. What are query operators in MongoDB? Explain with proper example.

Query operators in MongoDB are special keywords (prefixed with $) that allow you to build complex filter conditions when querying documents. They make it possible to express conditions like 'greater than', 'in this list', or 'not equal to' in a structured way.

1. Comparison Operators:

Operator	Meaning
$eq	Equal to
$ne	Not equal to
$gt	Greater than
$gte	Greater than or equal
$lt	Less than
$lte	Less than or equal
$in	Matches any value in an array
$nin	Does not match any value in an array

```
// Students with marks greater than 80
db.students.find({ marks: { $gt: 80 } })
```

```
// Students with marks between 60 and 90
db.students.find({ marks: { $gte: 60, $lte: 90 } })

// Students in either FSWD or DSA course
db.students.find({ course: { $in: ["FSWD", "DSA"] } })

// Students NOT in CSE or ECE branch
db.students.find({ branch: { $nin: ["CSE", "ECE"] } })
```

2. Logical Operators:

Operator	Meaning
$and	All conditions must be true
$or	At least one condition must be true
$not	Negates a condition
$nor	None of the conditions must be true

```
// $and: FSWD students with marks > 80
db.students.find({
  $and: [
    { course: "FSWD" },
    { marks: { $gt: 80 } }
  ]
})

// $or: Students in FSWD OR with marks > 90
db.students.find({
  $or: [
    { course: "FSWD" },
    { marks: { $gt: 90 } }
  ]
})

// $not: Students whose marks are NOT greater than 70
db.students.find({ marks: { $not: { $gt: 70 } } })
```

3. Element Operators:

```
// $exists: Find documents where 'email' field exists
db.students.find({ email: { $exists: true } })

// $type: Find documents where 'age' is a number
db.students.find({ age: { $type: "int" } })
```

Q53. Explain MongoDB comparison and logical operators with proper example.

MongoDB provides a rich set of operators to filter documents. Comparison operators compare field values, while logical operators combine multiple conditions. These are heavily used in real-world applications for filtering, searching, and analytics.

Sample Collection Setup:

```
db.products.insertMany([
  { name: "Laptop",  price: 55000, stock: 10, category: "Electronics" },
  { name: "Phone",   price: 15000, stock: 50, category: "Electronics" },
  { name: "Desk",    price: 8000,  stock: 5,  category: "Furniture"   },
  { name: "Chair",   price: 3500,  stock: 20, category: "Furniture"   },
  { name: "Monitor", price: 22000, stock: 0,  category: "Electronics" }
])
```

Comparison Operators — Examples:

```
// $eq: Find products in 'Furniture' category
db.products.find({ category: { $eq: "Furniture" } })

// $ne: Products NOT in Electronics
db.products.find({ category: { $ne: "Electronics" } })

// $gt / $lt: Price between 5000 and 25000
db.products.find({ price: { $gt: 5000, $lt: 25000 } })

// $in: Products that are either Laptop or Monitor
db.products.find({ name: { $in: ["Laptop", "Monitor"] } })
```

Logical Operators — Examples:

```
// $and: Electronics with price < 20000
db.products.find({
  $and: [
    { category: "Electronics" },
    { price: { $lt: 20000 } }
  ]
})  // Result: Phone (15000)

// $or: Products in Furniture OR price > 50000
db.products.find({
  $or: [
    { category: "Furniture" },
    { price: { $gt: 50000 } }
  ]
})  // Result: Laptop, Desk, Chair

// $nor: Neither Electronics NOR price < 5000
```

```
db.products.find({
  $nor: [
    { category: "Electronics" },
    { price: { $lt: 5000 } }
  ]
})  // Result: Desk (8000, Furniture)

// $not: Stock is NOT 0 (items in stock)
db.products.find({ stock: { $not: { $eq: 0 } } })
```

Operator Type	Operator	Example Use
Comparison	$gt, $lt, $gte, $lte	price: {$gt: 1000}
Comparison	$eq, $ne	category: {$eq: 'Electronics'}
Comparison	$in, $nin	name: {$in: ['A','B']}
Logical	$and	Both conditions true
Logical	$or	Either condition true
Logical	$nor	Neither condition true
Logical	$not	Negate a condition

Q54. What is Aggregation? Explain Aggregation in MongoDB with proper example.

Aggregation in MongoDB is a way to process and transform data through a pipeline of stages. Each stage receives documents from the previous stage, performs an operation, and passes results to the next. It is similar to SQL's GROUP BY, JOIN, and HAVING combined into one powerful tool.

Analogy:

Think of an assembly line in a factory. Raw materials (documents) enter the line, and at each station (stage) something happens — some are filtered out, some are reshaped, some are counted. The final product coming out at the end is your aggregated result.

Common Aggregation Stages:

Stage	Purpose
$match	Filter documents (like WHERE in SQL)
$group	Group documents and apply accumulators (SUM, AVG, COUNT)
$sort	Sort the results
$project	Include, exclude, or reshape fields
$limit	Limit the number of output documents
$skip	Skip N documents
$unwind	Deconstruct an array field into separate documents
$lookup	Perform a LEFT JOIN with another collection

Example 1 — Total revenue per customer:

```
db.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      totalRevenue: { $sum: { $multiply: ["$qty", "$price"] } },
      totalOrders: { $count: {} }
    }
  },
  { $sort: { totalRevenue: -1 } }
])
// Output: Raj: 132000, Priya: 100000, Sneha: 30000
```

Example 2 — Filter then group (only Laptop orders):

```
db.orders.aggregate([
  { $match: { product: "Laptop" } },
  {
    $group: {
      _id: "$customer",
      laptopSpend: { $sum: { $multiply: ["$qty", "$price"] } }
    }
  }
])
```

Example 3 — Average order price per product:

```
db.orders.aggregate([
  {
    $group: {
      _id: "$product",
      avgPrice: { $avg: "$price" },
      totalQty: { $sum: "$qty" }
    }
  },
  { $project: { _id: 1, avgPrice: 1, totalQty: 1 } }
])
```

Aggregation Pipeline Flow:

Documents --> [$match] --> [$group] --> [$sort] --> [$project] --> Result

Q55. How can we connect PHP with MongoDB? Explain in details.

PHP can connect to MongoDB using the official MongoDB PHP Driver (mongodb extension) combined with the MongoDB PHP Library. Together they provide a clean, object-oriented API to interact with MongoDB from PHP applications.

Step-by-Step Connection Process:

1. Install the MongoDB PHP Extension using PECL:

```
pecl install mongodb
```

2. Enable the extension in php.ini:

```
extension=mongodb.so
```

3. Install the MongoDB PHP Library via Composer for a higher-level API:

```
composer require mongodb/mongodb
```

4. Autoload and connect in PHP:

```
<?php
// Include Composer's autoloader
require 'vendor/autoload.php';

// Create a MongoDB client (connects to localhost:27017 by default)
$client = new MongoDB\Client("mongodb://localhost:27017");

// Select a database
$db = $client->studentDB;

// Select a collection
$collection = $db->students;

// Test connection: List all databases
foreach ($client->listDatabases() as $database) {
```

```
  echo $database->getName() . "\n";
}
?>
```

5. Connecting to MongoDB Atlas (Cloud):

```
<?php
require 'vendor/autoload.php';

// Replace <username>, <password>, <cluster> with your Atlas credentials
$uri =
"mongodb+srv://<username>:<password>@cluster0.abcde.mongodb.net/?retryWrites=tr
ue";
$client = new MongoDB\Client($uri);
$db = $client->myAppDB;
echo "Connected to MongoDB Atlas successfully!";
?>
```

Parameter	Description
mongodb://	Protocol for local connections
mongodb+srv://	Protocol for Atlas (cloud) connections
localhost:27017	Default host and port
username:password	Authentication credentials
dbName	Name of the target database

Error Handling:

```
<?php
require 'vendor/autoload.php';
try {
  $client = new MongoDB\Client("mongodb://localhost:27017");
  $db = $client->studentDB;
  echo "Connected successfully to MongoDB!";
} catch (MongoDB\Driver\Exception\Exception $e) {
  echo "Connection failed: " . $e->getMessage();
}
?>
```

Q56. Write a PHP code to perform CRUD Operations (insert, update, delete, read) with PHP & MongoDB.

The following complete PHP examples demonstrate all four CRUD operations using the MongoDB PHP Library. Each operation targets a 'students' collection in a 'studentDB' database.

Setup (common across all examples):

```
<?php
require 'vendor/autoload.php';
$client     = new MongoDB\Client("mongodb://localhost:27017");
$db         = $client->studentDB;
$collection = $db->students;
?>
```

1. CREATE — Insert Documents:

```
<?php
require 'vendor/autoload.php';
$collection = (new MongoDB\Client("mongodb://localhost:27017"))->studentDB-
>students;

// Insert one document
$result = $collection->insertOne([
  'name'       => 'Raj ',
  'enrollment' => '2302031800037',
  'semester'   => 6,
  'branch'     => 'CSE',
  'marks'      => 88
]);
echo "Inserted ID: " . $result->getInsertedId() . "\n";

// Insert multiple documents
$result = $collection->insertMany([
  ['name' => 'Priya Shah',   'marks' => 92, 'branch' => 'CE'],
  ['name' => 'Arjun Mehta', 'marks' => 75, 'branch' => 'CSE']
]);
echo "Inserted " . $result->getInsertedCount() . " documents\n";
?>
```

2. READ — Find Documents:

```
<?php
require 'vendor/autoload.php';
$collection = (new MongoDB\Client("mongodb://localhost:27017"))->studentDB-
>students;

// Find all documents
$cursor = $collection->find([]);
```

```
foreach ($cursor as $doc) {
  echo $doc["name"] . " - Marks: " . $doc["marks"] . "\n";
}

// Find with filter (marks > 80)
$cursor = $collection->find(['marks' => ['$gt' => 80]]);

// Find one document
$student = $collection->findOne(['name' => 'Raj ']);
echo "Found: " . $student["name"] . "\n";

// With projection (only name and marks)
$cursor = $collection->find(
  [],
  ['projection' => ['name' => 1, 'marks' => 1, '_id' => 0]]
);
?>
```

3. UPDATE — Modify Documents:

```
<?php
require 'vendor/autoload.php';
$collection = (new MongoDB\Client("mongodb://localhost:27017"))->studentDB-
>students;

// Update one document (set marks to 95)
$result = $collection->updateOne(
  ['name' => 'Raj '],          // filter
  ['$set' => ['marks' => 95]]       // update
);
echo "Modified: " . $result->getModifiedCount() . " document(s)\n";

// Update many documents (set semester to 6 for all CSE students)
$result = $collection->updateMany(
  ['branch' => 'CSE'],
  ['$set' => ['semester' => 6]]
);

// Upsert (insert if not found, update if found)
$collection->updateOne(
  ['name' => 'New Student'],
  ['$set' => ['name' => 'New Student', 'marks' => 70]],
  ['upsert' => true]
);
?>
```

4. DELETE — Remove Documents:

```
<?php
require 'vendor/autoload.php';
$collection = (new MongoDB\Client("mongodb://localhost:27017"))->studentDB-
>students;

// Delete one document
$result = $collection->deleteOne(['name' => 'Arjun Mehta']);
echo "Deleted: " . $result->getDeletedCount() . " document(s)\n";

// Delete many documents (all students with marks < 50)
$result = $collection->deleteMany(['marks' => ['$lt' => 50]]);
echo "Deleted: " . $result->getDeletedCount() . " document(s)\n";
?>
```

Q57. What is indexing? How is indexing implemented in MongoDB?

Indexing is a data structure technique that allows MongoDB to find documents quickly without scanning every document in a collection. Without an index, MongoDB must perform a collection scan — reading every document — which is slow for large datasets.

Analogy:

Think of an index in a textbook. Instead of reading every page to find 'Aggregation', you look at the index, which tells you to go directly to page 120. MongoDB indexes work the same way — they store a sorted reference to field values so queries jump directly to matching documents.

Types of Indexes in MongoDB:

Index Type	Description
Single Field	Index on one field (most common)
Compound	Index on multiple fields combined
Multikey	Index on array fields (each array element indexed)
Text	Full-text search index on string fields
Geospatial	For location-based queries ($near, $geoWithin)
Hashed	For hash-based sharding — even distribution
Unique	Ensures no two documents have the same value for the field
TTL (Time-To-Live)	Automatically deletes documents after a set time

Creating Indexes — Code Examples:

```
// Single field index (ascending = 1, descending = -1)
db.students.createIndex({ name: 1 })

// Unique index (no duplicate emails allowed)
db.students.createIndex({ email: 1 }, { unique: true })

// Compound index (sort by branch first, then marks descending)
db.students.createIndex({ branch: 1, marks: -1 })

// Text index for full-text search
db.articles.createIndex({ content: "text" })
db.articles.find({ $text: { $search: "MongoDB tutorial" } })

// TTL index (auto-delete session docs after 3600 seconds = 1 hour)
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })

// List all indexes on a collection
db.students.getIndexes()

// Drop an index
db.students.dropIndex({ name: 1 })

// Explain a query to check if index is being used
db.students.find({ name: "Raj" }).explain("executionStats")
```

Without Index	With Index
Collection scan (all docs read)	Index scan (only matching entries)
Slow for large collections	Fast even with millions of docs
O(n) time complexity	O(log n) time complexity
No extra storage needed	Extra storage for index data structure

Q58. How can data validation be performed in MongoDB?

Data validation in MongoDB ensures that documents inserted or updated in a collection meet defined rules. MongoDB supports schema validation using JSON Schema, which allows you to enforce data types, required fields, and value constraints directly at the database level.

Methods of Validation:

- JSON Schema Validation: The primary method using $jsonSchema — define expected field types, required fields, and allowed values.
- Query Expression Validation: Use regular MongoDB query operators inside the validator.
- Validation Levels and Actions: Control how strictly validation is enforced.

Setting Validation on Collection Creation:

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "enrollment", "marks"],
      properties: {
        name: {
          bsonType: "string",
          description: "Name must be a string and is required"
        },
        enrollment: {
          bsonType: "string",
          pattern: "^[0-9]{16}$",   // Must be 16 digits
          description: "Enrollment number must be 16 digits"
        },
        marks: {
          bsonType: "int",
          minimum: 0,
          maximum: 100,
          description: "Marks must be an integer between 0 and 100"
        },
        email: {
          bsonType: "string",
          pattern: "^.+@.+\..+$",
          description: "Must be a valid email address"
        }
      }
    }
  },
  validationAction: "error",  // Reject invalid documents ("warn" just logs)
  validationLevel: "strict"   // Apply to all inserts and updates
})
```

Adding Validation to Existing Collection:

```
db.runCommand({
  collMod: "students",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name"],
      properties: {
        name:  { bsonType: "string" },
        marks: { bsonType: "int", minimum: 0, maximum: 100 }
      }
    }
  }
})
```

Option	Values & Meaning
validationAction	'error' = reject; 'warn' = allow but log
validationLevel	'strict' = all ops; 'moderate' = existing docs exempt

Q59. Explain how to use the MongoDB Query API.

The MongoDB Query API is the complete set of operations, commands, and methods used to interact with MongoDB data. It includes CRUD operations, aggregation, text search, geospatial queries, and more — available through the MongoDB Shell, drivers, and Compass.

Core Components of the Query API:

- CRUD Operations: insertOne, insertMany, find, findOne, updateOne, updateMany, deleteOne, deleteMany
- Query Operators: Comparison ($gt, $lt), Logical ($and, $or), Element ($exists, $type), Array ($elemMatch, $size)
- Aggregation Pipeline: $match, $group, $sort, $project, $lookup, $unwind
- Text Search: Full-text search using text indexes and the $text operator
- Geospatial Queries: Location-based queries using $near, $geoWithin, $geoIntersects
- Update Operators: $set, $unset, $inc, $push, $pull, $addToSet

Key Query API Examples:

```
// --- FIND with multiple operators ---
db.products.find({
  $and: [
    { price: { $gte: 1000, $lte: 50000 } },
    { category: { $in: ["Electronics", "Appliances"] } },
    { stock: { $gt: 0 } }
  ]
}).sort({ price: 1 }).limit(10)

// --- UPDATE OPERATORS ---
db.students.updateOne(
  { name: "Raj" },
  {
    $set:   { semester: 6 },      // Set a field
    $inc:   { marks: 5 },         // Increment a number
    $unset: { tempField: "" },    // Remove a field
    $push:  { subjects: "FSWD" } // Add to an array
  }
)

// --- ARRAY OPERATORS ---
// Find documents where 'scores' array contains value > 90
db.students.find({ scores: { $elemMatch: { $gt: 90 } } })

// Find documents where 'tags' array has exactly 3 elements
db.articles.find({ tags: { $size: 3 } })

// --- TEXT SEARCH ---
db.articles.createIndex({ content: "text", title: "text" })
db.articles.find({ $text: { $search: "MongoDB tutorial beginner" } })

// --- GEOSPATIAL ---
db.places.createIndex({ location: "2dsphere" })
db.places.find({
  location: {
    $near: {
      $geometry: { type: 'Point', coordinates: [72.5714, 23.0225] },
      $maxDistance: 5000  // 5km radius
    }
  }
})
```

Q60. What is the purpose of MongoDB Charts?

MongoDB Charts is a data visualization tool built directly into MongoDB Atlas. It allows you to create visual dashboards — charts, graphs, and tables — directly from your MongoDB data without writing any code or exporting data to external tools.

Key Purposes:

- Visual Data Exploration: Turn raw MongoDB documents into meaningful bar charts, line graphs, scatter plots, heat maps, and more with a drag-and-drop interface.
- Real-time Dashboards: Charts automatically refresh as underlying MongoDB data changes, giving you a live view of your data.
- No ETL Required: Unlike traditional BI tools that need data exported to a data warehouse, MongoDB Charts queries your live MongoDB data directly.
- Embedded Charts: Embed charts in your web applications using an SDK, allowing end-users to see visual analytics inside your app.
- Collaboration: Share dashboards with team members, set permissions (view/edit), and embed charts in presentations or reports.
- Aggregation Integration: Under the hood, Charts uses MongoDB's aggregation pipeline to process and transform data for visualization.

Types of Charts Supported:

Chart Type	Best Used For
Bar / Column Chart	Comparing categories (sales per product)
Line Chart	Trends over time (daily signups)
Pie / Donut Chart	Proportion of a whole (market share)
Scatter Plot	Correlation between two variables
Heat Map	Intensity distribution across two dimensions
Data Table	Raw tabular view of aggregated data
Geo Map	Geographic distribution of data
Word Cloud	Frequency of text/tags in documents

How to Create a Chart (Process):

1. Log in to MongoDB Atlas and navigate to the Charts section.
2. Connect a data source (your Atlas cluster).
3. Create a new Dashboard and add a chart.
4. Select the collection (e.g., orders).

5. Drag fields to the X-axis, Y-axis, and Color/Group fields.
6. Choose the chart type (Bar, Line, Pie, etc.).
7. Apply filters and aggregation as needed.
8. Save and share the dashboard.

UNIT 5: Introduction to Laravel

Q61. What is Laravel? Explain its key features and benefits.

Laravel is a free, open-source PHP web application framework built by Taylor Otwell in 2011. It follows the MVC (Model-View-Controller) architectural pattern and is designed to make web development elegant, expressive, and enjoyable. Laravel comes with a rich set of built-in tools that eliminate repetitive boilerplate code.

Analogy:

If PHP is a toolbox with raw tools, Laravel is like a fully equipped kitchen — all tools are pre-arranged, labeled, and ready to use. You focus on cooking (building features), not on finding the right utensil.

Key Features:

- MVC Architecture: Clean separation of business logic (Model), presentation (View), and user input handling (Controller) making code organized and maintainable.
- Eloquent ORM: An expressive, ActiveRecord-style ORM for database interactions. Map database tables to PHP classes with no raw SQL needed.
- Blade Templating Engine: A powerful, lightweight template engine with directives like @if, @foreach, @extends, @yield for reusable layouts.
- Artisan CLI: A command-line tool to generate boilerplate code (models, controllers, migrations) with single commands like php artisan make:model.
- Migrations: Version control for your database schema. Create, modify, and rollback database tables using PHP code instead of raw SQL.
- Routing: Clean, expressive URL routing with support for RESTful resources, route grouping, middleware, and named routes.
- Middleware: Filter HTTP requests (authentication, CORS, logging) before they reach the application logic.
- Authentication & Authorization: Built-in scaffolding for user login, registration, and role-based access control using Gates and Policies.
- Queue System: Run time-consuming tasks (emails, image processing) in the background using queues and jobs.
- Testing Support: Built-in support for PHPUnit testing with helper methods for testing HTTP requests, databases, and more.

Feature	Benefit
Eloquent ORM	No raw SQL; intuitive database access
Blade Templates	Reusable, clean HTML templates
Artisan CLI	Rapid code generation
Migrations	Database version control
Built-in Auth	User authentication out of the box
Routing	Clean, readable URL definitions

Q62. Describe the Model-View-Controller (MVC) architecture. How does Laravel implement MVC?

MVC is a software design pattern that separates an application into three interconnected components. This separation makes code easier to manage, test, and scale by keeping different concerns isolated from each other.

The Three Components:

- Model: Represents the data layer. It handles database interactions — fetching, inserting, updating, and deleting records. The model is the single source of truth for the application's data.
- View: The presentation layer. It is what the user sees — HTML, CSS, and dynamic content rendered from data passed by the controller. Views should contain no business logic.
- Controller: The middleman. It receives user input (HTTP requests), calls the appropriate Model to get or save data, and then passes that data to the View to render a response.

MVC Request Flow:

```
User Browser
     |
     | HTTP Request (GET /students)
     v
[ROUTER]  -->  routes/web.php
     |
     v
[CONTROLLER]  -->  StudentController.php
     |  Calls Model
     v
[MODEL]  -->  Student.php (Eloquent)
     |  Queries Database
     v
[DATABASE]  -->  MySQL / MongoDB
     |  Returns data to Controller
     v
```

```
[CONTROLLER]  -->  passes $students to View
     |
     v
[VIEW]  -->  students/index.blade.php
     |
     v
HTML Response  -->  User Browser
```

How Laravel Implements MVC:

Component	Laravel Equivalent	Location
Model	Eloquent Model class	app/Models/
View	Blade Template (.blade.php)	resources/views/
Controller	Controller class	app/Http/Controllers/
Router	Route definitions	routes/web.php or api.php

Code Example:

```
// routes/web.php (Router)
Route::get('/students', [StudentController::class, 'index']);

// app/Http/Controllers/StudentController.php (Controller)
class StudentController extends Controller {
  public function index() {
    $students = Student::all();                          // Calls Model
    return view('students.index', compact('students')); // Passes to View
  }
}

// app/Models/Student.php (Model)
class Student extends Model {
  protected $fillable = ['name', 'enrollment', 'marks'];
}

{{-- resources/views/students/index.blade.php (View) --}}
@foreach($students as $student)
  <p>{{ $student->name }} - {{ $student->marks }}</p>
@endforeach
```

Q63. How do you install Laravel on a local machine? Explain the steps.

Installing Laravel requires PHP, Composer (PHP's package manager), and optionally a local server like XAMPP or Laravel's built-in development server. Here is the complete step-by-step process.

Prerequisites:

- PHP >= 8.1 installed
- Composer installed (getcomposer.org)
- Node.js and NPM (for frontend assets)
- A database server: MySQL, PostgreSQL, or SQLite

Installation Steps:

1. Verify PHP and Composer installation:

```
php -v       # Should show PHP 8.1 or higher
composer -v  # Should show Composer version
```

2. Create a new Laravel project using Composer:

```
composer create-project laravel/laravel myapp
```

3. Or use the Laravel Installer (faster):

```
composer global require laravel/installer
laravel new myapp
```

4. Navigate into the project folder:

```
cd myapp
```

5. Copy the .env.example file and configure environment:

```
cp .env.example .env
# Edit .env for your database settings:
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=myapp
DB_USERNAME=root
DB_PASSWORD=
```

6. Generate the application encryption key:

```
php artisan key:generate
```

7. Run database migrations (creates default tables):

```
php artisan migrate
```

8. Start the development server:

```
php artisan serve
# Server starts at: http://127.0.0.1:8000
```

Step	Command
Install	composer create-project laravel/laravel myapp
Key Generate	php artisan key:generate
Migrate DB	php artisan migrate
Run Server	php artisan serve

Q64. What is Composer, and why is it required for Laravel installation?

Composer is a dependency manager for PHP — similar to npm for Node.js or pip for Python. It allows you to define the libraries your project depends on and automatically downloads and manages them. Composer reads a composer.json file to know what packages are needed.

Why is Composer Required for Laravel?

- Laravel is Not a Single File: Laravel is composed of dozens of third-party and first-party packages (Symfony components, Eloquent, Blade, etc.). Composer downloads and links all of these automatically.
- Dependency Resolution: Composer ensures all packages are compatible with each other and resolves version conflicts automatically.
- Autoloading: Composer generates an autoload.php file that automatically loads PHP classes when needed — no manual require statements.
- Package Updates: Updating Laravel or any package is as simple as running composer update.
- Central Registry: Composer connects to Packagist (packagist.org), the main PHP package repository, to find and download packages.

Key Composer Commands:

```
# Create a new Laravel project
composer create-project laravel/laravel myapp

# Install a specific package
composer require guzzlehttp/guzzle

# Remove a package
composer remove guzzlehttp/guzzle

# Update all packages
composer update

# Install dependencies from composer.lock (production)
composer install
```

```
# Show installed packages
composer show
```

File	Purpose
composer.json	Defines required packages and version constraints
composer.lock	Records the exact version of every installed package (used in production for consistency)

Q65. What are Laravel Artisan commands? Provide examples of commonly used commands.

Artisan is Laravel's built-in command-line interface (CLI). It provides a collection of commands that help automate repetitive development tasks — like creating files, running migrations, clearing caches, and starting the development server. You run Artisan commands from your terminal inside the Laravel project directory.

How to Use Artisan:

```
php artisan              # Lists all available commands
php artisan help migrate # Shows help for a specific command
```

1. Application Commands:

```
php artisan serve          # Start development server (http://127.0.0.1:8000)
php artisan key:generate   # Generate APP_KEY in .env
php artisan env            # Show current environment
```

2. Make Commands (Code Generation):

```
php artisan make:model Student                   # Create a Model
php artisan make:model Student -m                # Create Model + Migration
php artisan make:controller StudentController    # Create Controller
php artisan make:controller StudentController --resource  # CRUD Resource
Controller
php artisan make:migration create_students_table # Create Migration file
php artisan make:seeder StudentSeeder            # Create Seeder
php artisan make:middleware CheckAge             # Create Middleware
php artisan make:request StoreStudentRequest     # Create Form Request
```

3. Database Commands:

```
php artisan migrate          # Run all pending migrations
php artisan migrate:rollback # Undo last migration batch
php artisan migrate:fresh    # Drop all tables and re-run migrations
php artisan db:seed          # Run database seeders
php artisan migrate --seed   # Migrate and seed together
```

4. Cache Commands:

```
php artisan cache:clear    # Clear application cache
php artisan config:clear   # Clear config cache
php artisan route:clear    # Clear route cache
php artisan view:clear     # Clear compiled views
php artisan optimize:clear # Clear all caches at once
```

5. Route Commands:

```
php artisan route:list  # List all registered routes
php artisan route:cache # Cache routes for faster loading
```

Command	Purpose
php artisan serve	Run local dev server
php artisan make:model X -m	Model + Migration
php artisan migrate	Run migrations
php artisan route:list	See all routes
php artisan optimize:clear	Clear all caches

Q66. Explain the directory structure of a Laravel project in details.

When you create a new Laravel project, it comes with a well-organized directory structure. Each folder has a specific, dedicated purpose, making it easy to locate any file in the application.

```
myapp/
├── app/                    # Core application code
│   ├── Http/
│   │   ├── Controllers/    # Controller classes
│   │   ├── Middleware/     # Request filtering middleware
│   │   └── Requests/       # Form validation request classes
│   ├── Models/             # Eloquent model classes
│   └── Providers/          # Service providers (app bootstrapping)
├── bootstrap/              # App bootstrapping & cache
├── config/                 # Configuration files (app, db, mail, etc.)
├── database/
│   ├── migrations/         # Database schema version files
│   ├── seeders/            # Seed files for dummy data
│   └── factories/          # Model factories for testing
├── public/                 # Web server document root
│   └── index.php           # Entry point for all HTTP requests
├── resources/
│   ├── views/              # Blade template files (.blade.php)
│   ├── css/                # Raw CSS source files
│   └── js/                 # Raw JS source files
├── routes/
│   ├── web.php             # Web (browser) routes
│   ├── api.php             # API routes (stateless)
│   └── console.php         # Artisan command routes
├── storage/                # Logs, cache, sessions, uploaded files
├── tests/                  # Unit and feature test files
├── vendor/                 # Composer dependencies (never edit manually)
├── .env                    # Environment configuration (secrets)
├── artisan                 # Artisan CLI entry point
├── composer.json           # PHP dependencies
└── package.json            # Node.js dependencies
```

Directory/File	Purpose
app/Models/	Eloquent database models
app/Http/Controllers/	Handle HTTP requests
resources/views/	Blade HTML templates
routes/web.php	Define URL routes
database/migrations/	Database schema changes
public/index.php	Single entry point for all requests
.env	Environment secrets (DB password, API keys)
vendor/	All installed packages (via Composer)

Q67. Explain the concept of routing in Laravel.

Routing in Laravel maps HTTP requests (URLs) to specific actions — usually controller methods or closures. When a user visits a URL in a Laravel application, the router reads the URL and method (GET, POST, etc.) and decides which code to execute. All routes are defined in the routes/ directory.

Types of Routes:

- Web Routes (routes/web.php): For browser-based routes that use sessions, cookies, and CSRF protection.
- API Routes (routes/api.php): For stateless API endpoints (no sessions). Automatically prefixed with /api.

Basic Route Definitions:

```
// routes/web.php

// GET route with closure
Route::get('/home', function () {
  return view('home');
});

// GET route to a controller method
Route::get('/students', [StudentController::class, 'index']);

// POST route
Route::post('/students', [StudentController::class, 'store']);

// Route with parameter
Route::get('/student/{id}', function ($id) {
  return "Student ID: " . $id;
```

```
});

// Named route
Route::get('/about', function () {
  return view('about');
})->name('about');
// Usage: route('about') --> generates /about URL

// Route group with prefix and middleware
Route::prefix('admin')->middleware('auth')->group(function () {
  Route::get('/dashboard', [AdminController::class, 'dashboard']);
  Route::get('/users',     [AdminController::class, 'users']);
});

// RESTful Resource Route (auto-creates 7 CRUD routes)
Route::resource('students', StudentController::class);
```

Resource Route Breakdown:

HTTP Method	URL	Controller Method
GET	/students	index()
GET	/students/create	create()
POST	/students	store()
GET	/students/{id}	show()
GET	/students/{id}/edit	edit()
PUT/PATCH	/students/{id}	update()
DELETE	/students/{id}	destroy()

Q68. What are Controllers in Laravel, and how do they work?

Controllers in Laravel are PHP classes that handle HTTP requests. They act as the middleman between the Router and the Model. When a user makes a request to a URL, the router calls a specific method in a controller. The controller fetches data using Models and returns a response — usually a View.

Creating a Controller:

```
# Basic controller
php artisan make:controller StudentController

# Resource controller (generates all 7 CRUD methods automatically)
php artisan make:controller StudentController --resource
```

Basic Controller Example:

```php
<?php
// app/Http/Controllers/StudentController.php
namespace App\Http\Controllers;

use App\Models\Student;
use Illuminate\Http\Request;

class StudentController extends Controller {

  // Show list of all students
  public function index() {
    $students = Student::all();
    return view('students.index', compact('students'));
  }

  // Show form to add a new student
  public function create() {
    return view('students.create');
  }

  // Save new student to database
  public function store(Request $request) {
    $request->validate([
      'name'  => 'required|string|max:100',
      'marks' => 'required|integer|min:0|max:100',
    ]);
    Student::create($request->all());
    return redirect('/students')->with('success', 'Student added!');
  }

  // Update a student record
  public function update(Request $request, $id) {
    $student = Student::findOrFail($id);
```

```
    $student->update($request->all());
    return redirect('/students');
  }

  // Delete a student
  public function destroy($id) {
    Student::findOrFail($id)->delete();
    return redirect('/students');
  }
}

// routes/web.php — connect routes to controller
Route::resource('students', StudentController::class);
```

Q69. How do you pass data from a Controller to a View? Provide an example.

In Laravel's MVC pattern, controllers retrieve data from models and then pass it to views for display. There are three main ways to pass data from a controller to a Blade view.

Method 1: compact() — Most Common:

```
public function index() {
  $students = Student::all();
  $title    = "Student List";
  return view('students.index', compact('students', 'title'));
  // compact() creates array: ['students' => $students, 'title' => $title]
}
```

Method 2: with() — Chained:

```
public function index() {
  $students = Student::all();
  return view('students.index')
    ->with('students', $students)
    ->with('title', 'Student List');
}
```

Method 3: Array — Explicit:

```
public function index() {
  return view('students.index', [
    'students' => Student::all(),
    'title'    => 'Student List',
    'count'    => Student::count(),
  ]);
}
```

Accessing Data in the Blade View:

```
{{-- resources/views/students/index.blade.php --}}
<!DOCTYPE html>
<html>
<head><title>{{ $title }}</title></head>
<body>
  <h1>{{ $title }}</h1>
  <p>Total Students: {{ $count }}</p>
  <table border="1">
    <tr><th>Name</th><th>Marks</th></tr>
    @foreach($students as $student)
    <tr>
      <td>{{ $student->name }}</td>
      <td>{{ $student->marks }}</td>
    </tr>
    @endforeach
  </table>
</body>
</html>
```

Q70. Explain the concept of Blade Templating in Laravel. What are its benefits?

Blade is Laravel's built-in template engine. It allows you to write clean, readable HTML templates with special Blade directives that get compiled into plain PHP. Blade templates are stored in resources/views/ with a .blade.php extension and are automatically cached for performance.

Key Blade Directives:

```
{{-- Display a variable (auto-escaped for XSS safety) --}}
{{ $name }}

{{-- Display raw/unescaped HTML --}}
{!! $htmlContent !!}

{{-- Conditionals --}}
@if($marks >= 60)
  <p>Pass</p>
@elseif($marks >= 40)
  <p>Borderline</p>
@else
  <p>Fail</p>
@endif

{{-- Loops --}}
@foreach($students as $student)
  <p>{{ $student->name }}</p>
@endforeach
```

```
{{-- Loop $loop variable gives metadata --}}
@foreach($students as $student)
  @if($loop->first) <p>First student!</p> @endif
  <p>{{ $loop->index }}: {{ $student->name }}</p>
@endforeach
```

Template Inheritance (Master Layout):

```
{{-- resources/views/layouts/app.blade.php --}}
<!DOCTYPE html>
<html>
  <head><title>@yield('title')</title></head>
  <body>@yield('content')</body>
</html>

{{-- Child view that extends the layout --}}
@extends('layouts.app')
@section('title', 'Student List')
@section('content')
  <h1>All Students</h1>
  @foreach($students as $s)
    <p>{{ $s->name }}</p>
  @endforeach
@endsection

{{-- CSRF token for forms --}}
<form method="POST" action="/students">
  @csrf
  <input type="text" name="name">
  <button type="submit">Add</button>
</form>
```

Benefits of Blade:

- Clean Syntax: @if, @foreach, @extends are much cleaner than <?php if(): ?>
- Template Inheritance: Avoid code duplication by defining a master layout and extending it in child views.
- Auto-escaping: {{ $var }} automatically escapes HTML entities, preventing XSS attacks.
- Reusable Components: @include and @component allow reusable template pieces like navbars and footers.
- Performance: Blade templates are compiled to cached PHP — no performance overhead in production.

Q71. How do you retrieve input data from a form submission using Laravel Request?

In Laravel, when a user submits a form, the data is sent as an HTTP request. The Illuminate\Http\Request object provides all methods needed to access form data, files, headers, and more. The Request object is automatically injected into controller methods.

Step 1 — Create the HTML Form (View):

```
{{-- resources/views/students/create.blade.php --}}
<form method="POST" action="/students">
  @csrf
  <label>Name:</label>
  <input type="text" name="name" value="{{ old('name') }}">
  <label>Marks:</label>
  <input type="number" name="marks" value="{{ old('marks') }}">
  <label>Email:</label>
  <input type="email" name="email">
  <button type="submit">Add Student</button>
</form>
```

Step 2 — Define the Route:

```
Route::post('/students', [StudentController::class, 'store']);
```

Step 3 — Access Input in Controller:

```
<?php
use Illuminate\Http\Request;

class StudentController extends Controller {
  public function store(Request $request) {

    // Retrieve individual fields
    $name  = $request->input('name');
    $marks = $request->input('marks');

    // Shorthand using dynamic property
    $name = $request->name;

    // Get all input as array
    $allData = $request->all();

    // Get only specific fields
    $data = $request->only(['name', 'marks']);

    // Get all except a specific field
    $data = $request->except(['_token']);

    // Check if a field exists
```

```
    if ($request->has('email')) { /* email was submitted */ }

    // With validation
    $validated = $request->validate([
      'name'  => 'required|string|max:100',
      'marks' => 'required|integer|min:0|max:100',
      'email' => 'required|email|unique:students',
    ]);

    Student::create($validated);
    return redirect('/students')->with('success', 'Student added
successfully!');
  }
}
```

Q72. How do you create and handle Forms in Laravel?

Building forms in Laravel involves creating Blade views with HTML forms, defining routes that receive form submissions, and writing controller methods that validate and process the data. Laravel provides CSRF protection, validation, error display, and session flashing out of the box.

Step 1: Routes:

```
Route::get('/students/create', [StudentController::class, 'create']); // Show
form
Route::post('/students',        [StudentController::class, 'store']);  //
Handle submission
```

Step 2: Controller:

```
<?php
class StudentController extends Controller {

  public function create() {
    return view('students.create');
  }

  public function store(Request $request) {
    $validated = $request->validate([
      'name'       => 'required|string|max:100',
      'enrollment' => 'required|string|size:16|unique:students',
      'marks'      => 'required|integer|between:0,100',
      'email'      => 'required|email|unique:students',
    ]);
    Student::create($validated);
    return redirect('/students')->with('success', 'Student registered!');
  }
}
```

Step 3: Blade View (Form with Error Display):

```
{{-- resources/views/students/create.blade.php --}}
<!DOCTYPE html><html><head><title>Add Student</title></head><body>

@if(session('success'))
  <p style="color: green;">{{ session('success') }}</p>
@endif

<form method="POST" action="/students">
  @csrf
  <div>
    <label>Name:</label>
    <input type="text" name="name" value="{{ old('name') }}">
    @error('name') <span style="color:red">{{ $message }}</span> @enderror
  </div>
  <div>
    <label>Enrollment No:</label>
    <input type="text" name="enrollment" value="{{ old('enrollment') }}">
    @error('enrollment') <span style="color:red">{{ $message }}</span>
@enderror
  </div>
  <div>
    <label>Marks:</label>
    <input type="number" name="marks" value="{{ old('marks') }}">
    @error('marks') <span style="color:red">{{ $message }}</span> @enderror
  </div>
  <button type="submit">Register</button>
</form>
</body></html>
```

Key Laravel Form Features:

- @csrf: Auto-generates a hidden CSRF token to prevent cross-site request forgery attacks.
- old('field'): Re-populates form fields with previous values when validation fails.
- @error('field'): Displays validation error messages next to the relevant field.
- session('success'): Displays one-time flash messages after a redirect.

Q73. How do you create and manipulate Laravel Collections? Provide examples.

Laravel Collections are a powerful wrapper around PHP arrays provided by the Illuminate\Support\Collection class. They offer a fluent, chainable API with over 100 methods for filtering, transforming, sorting, and aggregating data — making array manipulation much cleaner and more readable than raw PHP.

Creating a Collection:

```php
<?php
use Illuminate\Support\Collection;

// Create from an array
$collection = collect([10, 20, 30, 40, 50]);

// Create from array of associative arrays
$students = collect([
  ['name' => 'Raj',   'marks' => 88, 'branch' => 'CSE'],
  ['name' => 'Priya', 'marks' => 92, 'branch' => 'CE'],
  ['name' => 'Arjun', 'marks' => 75, 'branch' => 'CSE'],
  ['name' => 'Sneha', 'marks' => 55, 'branch' => 'IT'],
]);
```

Common Collection Methods:

```php
// filter() - Keep only students with marks > 80
$topStudents = $students->filter(fn($s) => $s['marks'] > 80);
// Result: Raj (88), Priya (92)

// map() - Transform each item
$names = $students->map(fn($s) => strtoupper($s['name']));
// Result: ['RAJ', 'PRIYA', 'ARJUN', 'SNEHA']

// where() - Filter by field value
$cseStudents = $students->where('branch', 'CSE');

// sortBy() / sortByDesc()
$sorted = $students->sortByDesc('marks');

// pluck() - Extract one field from all items
$allNames = $students->pluck('name');
// Result: ['Raj', 'Priya', 'Arjun', 'Sneha']

// first() / last()
$top = $students->sortByDesc('marks')->first(); // Priya (92)

// sum() / avg() / max() / min()
$totalMarks = $students->sum('marks');   // 310
```

```
$avgMarks   = $students->avg('marks');   // 77.5
$maxMarks   = $students->max('marks');   // 92

// groupBy() - Group by field
$byBranch = $students->groupBy('branch');
// Result: ['CSE' => [Raj, Arjun], 'CE' => [Priya], 'IT' => [Sneha]]

// Chaining multiple methods
$result = $students
  ->filter(fn($s) => $s['branch'] === 'CSE')
  ->sortByDesc('marks')
  ->pluck('name');
// Result: ['Raj', 'Arjun']
```

Collections from Eloquent: Eloquent automatically returns Collection objects from queries like Student::all() — so all collection methods work directly on database results with no extra step.

Q74. What is Eloquent ORM in Laravel? How does it simplify database interactions? Discuss the benefits of it.

Eloquent is Laravel's built-in Object-Relational Mapping (ORM) tool. It allows you to interact with your database using PHP classes and objects instead of writing raw SQL queries. Each database table is represented by a corresponding Eloquent Model class, and each row in that table is represented as an instance of that model.

Analogy:

Instead of saying 'SELECT * FROM students WHERE id = 5' in SQL, Eloquent lets you say Student::find(5). The ORM translates your PHP code into SQL automatically and returns the result as a PHP object.

Setting Up an Eloquent Model:

```
# Create model (with migration)
php artisan make:model Student -m

<?php
// app/Models/Student.php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Student extends Model {
  // The table name (Laravel auto-guesses 'students' from 'Student')
  protected $table = 'students';

  // Fields that can be mass-assigned (for create/update)
```

```
  protected $fillable = ['name', 'enrollment', 'marks', 'branch', 'email'];

  // Timestamps (created_at, updated_at) — enabled by default
  public $timestamps = true;
}
```

CRUD with Eloquent:

```
<?php
// CREATE
Student::create(['name' => 'Raj', 'marks' => 88, 'branch' => 'CSE']);

// READ
$all     = Student::all();                              // All students
$one     = Student::find(5);                           // By primary key
$student = Student::where('marks', '>', 80)->get();    // With condition
$first   = Student::where('branch', 'CSE')->first();   // First match
$student = Student::findOrFail(99);                    // 404 if not found

// UPDATE
$student = Student::find(5);
$student->marks = 95;
$student->save();
// Or: Student::where('branch', 'CSE')->update(['semester' => 6]);

// DELETE
Student::find(5)->delete();
Student::where('marks', '<', 40)->delete();
```

Eloquent Relationships:

```
<?php
// One student has many courses
class Student extends Model {
  public function courses() {
    return $this->hasMany(Course::class);
  }
}

// Usage
$student = Student::find(1);
$courses = $student->courses; // SELECT * FROM courses WHERE student_id = 1
```

Benefit	Explanation
No raw SQL	Write PHP instead of SQL — cleaner and safer
Mass assignment	Create records with one create() call
Relationships	hasOne, hasMany, belongsTo, manyToMany — easy joins
Scopes	Reusable query filters as model methods
Events & Observers	Run code on model events (creating, updating, deleting)
Collections	Results are automatically returned as Collections
Soft Deletes	Mark records as deleted without actually removing them

Q75. What are Packages in Laravel? How to install it? Discuss Popular Laravel Packages.

Packages in Laravel are reusable pieces of code that add specific functionality to your application. Just like plugins for WordPress or npm packages for Node.js, Laravel packages let you add features like authentication, payment processing, image handling, and more without building everything from scratch. Most Laravel packages are installed via Composer.

How to Install a Laravel Package:

1. Find the package on Packagist (packagist.org) or the package's GitHub page.
2. Install using Composer:
```
composer require vendor/package-name
```
3. Publish the package's configuration file (if applicable):
```
php artisan vendor:publish --provider="Vendor\Package\ServiceProvider"
```
4. Run migrations if the package adds database tables:
```
php artisan migrate
```
5. Configure the package in config/ files as needed.

Popular Laravel Packages:

Package	Purpose
laravel/breeze	Minimal auth scaffolding (login, register, reset password)
laravel/sanctum	API token authentication for SPAs and mobile apps
laravel/passport	Full OAuth2 server for API authentication
spatie/laravel-permission	Role and permission management (RBAC)
barryvdh/laravel-debugbar	Developer toolbar for debugging queries and requests
intervention/image	Image processing (resize, crop, watermark, convert)
maatwebsite/excel	Import and export Excel / CSV files
spatie/laravel-medialibrary	Associate files and images with Eloquent models
darkaonline/l5-swagger	Auto-generate API documentation with Swagger/OpenAPI
laravel/telescope	Debug assistant — inspect requests, jobs, emails, cache
guzzlehttp/guzzle	HTTP client for making API requests to external services
spatie/laravel-backup	Automated database and file backups

Example — Installing and Using spatie/laravel-permission:

```
# Install
composer require spatie/laravel-permission

# Publish config and migrations
php artisan vendor:publish --
provider="Spatie\Permission\PermissionServiceProvider"

# Run migrations (creates roles, permissions, and pivot tables)
php artisan migrate

<?php
// Add HasRoles trait to User model
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable {
  use HasRoles;
}
```

```
// Usage in controller or seeder
$user = User::find(1);
$user->assignRole('admin');
$user->givePermissionTo('edit articles');

// Check in controller or middleware
if ($user->hasRole('admin')) {
  // Admin-only logic
}

// Check in Blade view
@role('admin')
  <a href="/admin">Admin Panel</a>
@endrole
```